



RPCA Cyber Club
Faculty of Forensic Science
Royal Police Cadet Academy

Cybersecurity

Chapter 6

Cryptography

Asst.Prof.Pol.Lt.Col.Dr. Wongyos Keardsri (Bank)

Lecturer (Level 3) | Faculty of Forensic Science | Royal Police Cadet Academy | Thailand

Email: wongyos@gmail.com | Line ID: wongyos | Phone/WhatsApp: +6689-599-3490

Facebook: facebook.com/wongyos

Copyright © 2026

Introduction to Cryptography

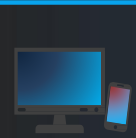




Definition of Cryptography

- **Cryptography** or **cryptology** (ancient **Greek**) is the practice and study of **techniques** for **securing communication** and **data**
- The **cryptography method** is the practice of concealing information using a **secret code** so that **only** the intended **recipient** can **read** it
- **Cryptography** is a crucial aspect of **cybersecurity**, ensuring confidentiality, integrity, authenticity, and non-repudiation of information





Definition of Cryptography (Cont)

- The pronunciation is /krip' tägrəfē/
- The **art of cryptography** has been used to **code messages** for **thousands of years** and continues to be used in bank cards, computer passwords, and ecommerce
- **Modern cryptography** exists at the intersection of the disciplines of **mathematics, computer science, electrical engineering, communication science, and physics**

Dictionary

Definitions from [Oxford Languages](#) · [Learn more](#)



crypt·tog·ra·phy

/krip' tägrəfē/

noun

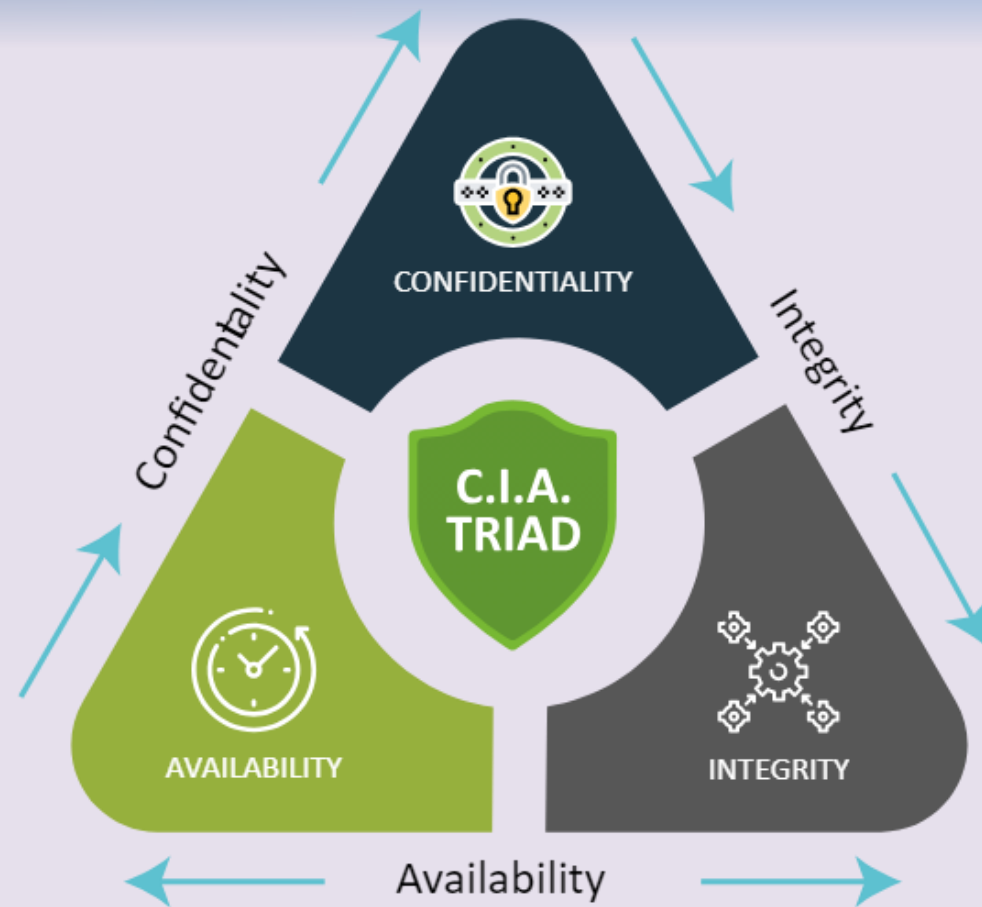
the art of writing or solving codes.

/krip' tägrəfē/



Definition of Cryptography (Cont)

- **Applications of cryptography** include electronic commerce, chip-based payment cards, digital currencies, computer passwords, and military communications
- The **CIA Triad—Confidentiality, Integrity, and Availability**—is a guiding model in cryptography and information security





Definition of Cryptography (Cont)

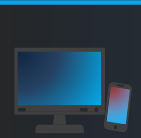
- The **CIA Triad (Cont)**
 - **Confidentiality** refers to protecting information from unauthorized access
 - **Integrity** means data are trustworthy, complete, and have not been accidentally altered or modified by an unauthorized user
 - **Availability** means data are accessible when you need them



Process of Cryptography

- Cryptography has **two** main processes, **encryption** and **decryption**
 - **Encryption**: the process of converting **plaintext** into an **unreadable format (ciphertext)** using a cryptographic algorithm and a key
 - **Decryption**: the process of converting **ciphertext** back into a **readable format (plaintext/cleartext)** using a cryptographic algorithm and the correct key





Process of Cryptography (Cont)

Encryption Formula

$$C = E(P, K)$$

Where:

C = Ciphertext

E = Encryption function

P = Plaintext

K = Encryption key

Decryption Formula

$$P = D(C, K)$$

Where:

P = Plaintext

D = Decryption function

C = Ciphertext

K = Decryption key



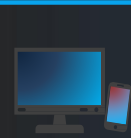
Types of Cryptography

■ Data Encoding and Decoding

- Uses a **basic encryption** and **decryption** method **traditionally** employed to ensure data confidentiality
- This approach typically **does not** involve the **use** of **cryptographic keys**
- **Example:** Binary, Octal, Hexadecimal, Base64 (Base families), ROT13, Morse, Braille

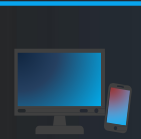
■ Symmetric Cryptography (Secret Key Cryptography)

- Uses the **same key** (**one key**) for encryption and decryption
- This approach is **fast** but requires **secure key distribution**
- **Example:** AES (Advanced Encryption Standard), DES (Data Encryption Standard)



Types of Cryptography (Cont)

- **Asymmetric Cryptography (Public Key Cryptography)**
 - Uses a **pair of keys** (two key): a **public key** for encryption and a **private key** for decryption
 - This approach is **slower** but provides better **security for key exchange**
 - **Example:** RSA, ECC (Elliptic Curve Cryptography)
- **Hashing (Message Digests)**
 - Converts input data into a **fixed-length hash value** (it **can encrypt** but **cannot decrypt**)
 - Used for integrity **verification** (e.g., digital signatures, password hashing)
 - **Example:** SHA-256, MD5 (deprecated due to vulnerabilities)



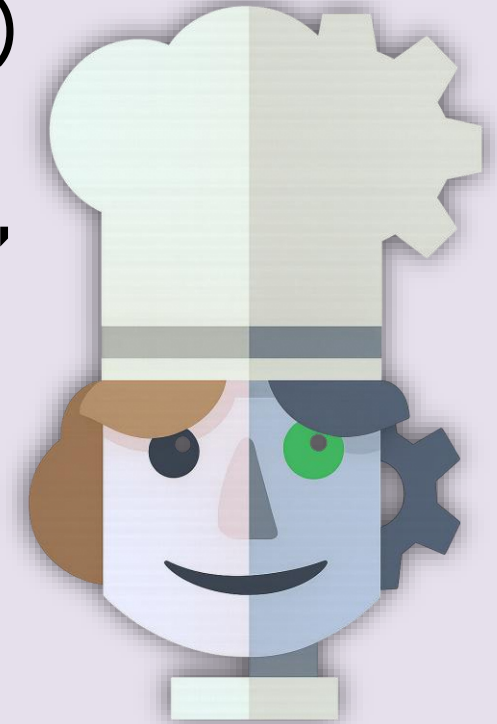
Applications of Cryptography

- **Data Encryption:** Protects sensitive information in storage and transmission
- **Digital Signatures:** Ensures authenticity and integrity in documents and transactions
- **Secure Communication:** Used in SSL/TLS protocols for safe web browsing
- **Blockchain & Cryptocurrency:** Ensures secure and tamper-proof transactions
- **Password Security:** Hashing and salting techniques prevent password leaks



Cryptography Tool: CyberChef

- **CyberChef** is a powerful, **open-source web-based** data processing tool developed by **GCHQ** (UK's intelligence agency)
- **CyberChef** allows users to perform a wide range of **operations** on data — such as **encoding, decoding, encryption, decryption, analysis, forensics**, and data **transformation** — through an intuitive drag-and-drop interface
- There are around **200+** useful **operations** in CyberChef, including a simple encoding like XOR or Base64, more complex encryption like AES, DES and Blowfish
- Official site: <https://gchq.github.io/CyberChef/>



Cryptography Tool: CyberChef (Cont)

The screenshot displays the CyberChef web application interface. The browser address bar shows `gchq.github.io/CyberChef/`. The interface is divided into four main sections, each highlighted with a red box and a blue callout label:

- Operations:** Located on the left, it contains a search bar and a list of operations including "To Base64", "From Base64", "To Hex", "From Hex", "To Hexdump", and "From Hexdump". A blue star icon is next to the "Favourites" header.
- Recipe:** The central area for building a sequence of operations. It features a "BAKE!" button and an "Auto Bake" checkbox.
- Input:** The top right section for pasting or uploading data to be processed.
- Output:** The bottom right section for viewing the results of the processing, including a "Raw Bytes" view and a "1ms" timer.



Cryptography Tool: CyberChef (Cont)

The screenshot shows the CyberChef web interface at `gchq.github.io/CyberChef/`. A red rectangular box highlights the "Download CyberChef" link, which includes a download icon. To the right of this link, a green banner states "Last build: 8 days ago - Version 10 is here! Read about t...". Below the banner, the interface is divided into three main sections: "Operations" (containing a "Search..." input field and a "Favourites" section with a star icon), "Recipe" (containing icons for undo, redo, and delete), and "Input" (containing a text input field). A blue callout bubble with a pointer directed at the "Download CyberChef" link contains the text "Download for offline mode".



Cryptography Tool: CyberChef (Cont)

The screenshot shows the CyberChef web application interface. The left sidebar contains a 'Recipe' section with a 'Magic' button highlighted by a red box. A blue callout bubble points to the 'Magic' button with the text 'Use the "Magic" for auto-detect encoding'. The main area is divided into 'Input' and 'Output' sections. The 'Input' section has a red box around the text 'QmFuayBXb25neW9z' with a blue callout bubble saying 'Input code for detection'. The 'Output' section shows a table with three columns: 'Recipe (click to load)', 'Result snippet', and 'Properties'. The first row of the table has a red box around the 'Recipe' cell containing the code 'From_Base64('A-Za-z0-9+/'',true,false)', a blue callout bubble pointing to it saying 'Base64', and a red box around the 'Result snippet' cell containing the text 'Bank Wongyos', with a blue callout bubble pointing to it saying 'Result'. The 'Properties' column shows 'Valid UTF8' and 'Entropy: 3.25'.

Recipe

Magic

Depth 3

☐ Int

☐ Extensive language support

Input

QmFuayBXb25neW9z

Output

Base64

Result

Recipe (click to load)	Result snippet	Properties
From_Base64('A-Za-z0-9+/'',true,false)	Bank Wongyos	Valid UTF8 Entropy: 3.25



Cryptography Tool: dCode.fr

- dCode.fr is a free universal site/tool for encoding and decoding messages, cheating on letter games, solving puzzles, geocaches and treasure hunts, etc.
- dCode.fr is a comprehensive online platform offering over 900 tools designed to assist with cryptography, steganography, mathematics, puzzles, and more
- It's widely utilized by enthusiasts of escape rooms, geocaching, CTF competitions, and puzzle-solving communities
- Official site: <https://www.dcode.fr/>





Search for a tool

★ SEARCH A TOOL ON DCODE BY KEYWORDS:

e.g. type 'boolean'

★ [BROWSE THE FULL DCODE TOOLS' LIST](#)

Results









dCode Brainfuck

+++++++[>+>+>++++>+++++++
 <<<<-]>>>>.<--.>+++++++.-----+.
 <<+,>.>++++++++.-----
 -.+++++.++++.-----.+++++++-.
 -----.+++++. .

Brainfuck - dCode

Tag(s) : Programming Language

Share



dCode and more

BRAINFUCK

Informatics - Programming Language - Brainfuck

BRAINFUCK INTERPRETER

★ BRAINF*CK CODE TO INTERPRET

```
+++++++ [ >+++++++>+++++++<<- ] >++++. -.
>+++++. <+. +.
```

★ ARGUMENT

★ SHOW MEMORY STATE ☒

Decode

► EXECUTE

BRAINFUCK ENCODER

★ PLAINTEXT TO CODE IN BRAINF**K (?)

dCode Brainfuck

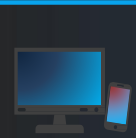
★ ADD A SEPARATOR BETWEEN INSTRUCTIONS ☐

▶ ENCRYPT

Encode

Summary

- Brainfuck Interpreter
- Brainfuck Encoder
- What is Brainfuck?
(Definition)
- How does Brainfuck work?
- How to encrypt using
Brainfuck code?
- How to encrypt using
Brainfuck Shortcut code?
- How to decrypt Brainfuck
code?
- How to decrypt Brainfuck
Shortcut code?
- How to recognize Brainfuck
coded text?
- What is the memory state?
- What are the variants of the
Brainfuck code?
- What is Brainfuck for?
- When was Brainfuck



Cryptography Tool: dCode.fr (Cont)

Search for a tool

Use **"Cipher Identifier"**
for **auto-detect**
encoding

KEYWORDS:

TOOLS' LIST



dCode's analysis tools to investigate:

Warning The text has a small length, this can affect the quantity and reliability of the results (see FAQ)

Warning Few or no significant results (see FAQ)

↑↓	↑↓
Base64 Coding	☒
Base 58	☐
Base62 Encoding	☐
Substitution Cipher	☐

Cipher Identifier - dCode

ENCRYPTED MESSAGE IDENTIFIER

CIPHERTEXT TO RECOGNIZE

QmFuayBXb25new9z

Input code for
detection

★ CLUES/KEYWORDS (IF ANY)

▶ ANALYZE

See also: [Frequency Analysis](#) – [Index of Coincidence](#)

SYMBOLS IDENTIFIER

▢ Go to: [Symbols Cipher List](#)

Answers to Questions (FAQ)

What is a cipher identifier? (Definition)

An encryption detector is a computer tool designed to recognize encryption/encoding from a text message. The detector performs cryptanalysis, examines various features of the text, such as letter



Cryptography Tool: dCode.fr (Cont)

Search for a tool

Use “Hash Identifier” for auto-detect hashing

Results

The following hash algorithms generate a fingerprint similar to 08da6d15c78e19c

↑↓

MD2

MD5

MD4

Double MD5

HASH IDENTIFIER

Cryptography › Modern Cryptography › Hash Identifier

HASH IDENTIFIER

★ HASH/FINGERPRINT

08da6d15c78e8b5c440a0b339ac3419c

▶ ANALYZE

See also: Cipher Identifier – Hash Function

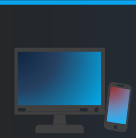
Input code for detection

Answers to Questions (FAQ)

What is a Hash identifier? (Definition)

A hash identifier is a tool that allows you to recognize the type of hash from a list of known **hash functions**. Given an unknown hash, the identifier determines which **hash function** was likely used to generate it.

There are hundreds of **hashing algorithms**, the majority return a hash as a number, usually stored in **hexadecimal** format. But the length of this



Cryptography Tool: OpenSSL

- **OpenSSL** is a powerful **command-line toolkit** and **library** for the implementation of the **SSL** and **TLS protocols** as well as **general-purpose cryptography**
- It's widely used on systems like **Kali Linux** for penetration testing, **encryption**, certificate generation, and more

OpenSSL
Cryptography and SSL/TLS Toolkit



Cryptography Tool: OpenSSL (Cont)

```
root@kali:~# echo 'Cryptography' | openssl enc -base64
Q3J5cHRvZ3JhcGh5Cg==
root@kali:~# echo 'Cryptography' | openssl enc -e -base64
Q3J5cHRvZ3JhcGh5Cg==
root@kali:~# echo 'Q3J5cHRvZ3JhcGh5Cg==' | openssl enc -d
-base64
Cryptography
root@kali:~# echo 'Cryptography' | openssl enc -des -base64 -K e0e0e0e0f1f1f1f1 -iv
e0e0e0e0abababab
NYC/82QHAY44n5n3ti9WBA==
root@kali:~# echo 'NYC/82QHAY44n5n3ti9WBA==' | openssl enc
-d -des -base64 -K e0e0e0e0f1f1f1f1 -iv e0e0e0e0abababab
Cryptography
root@kali:~# echo 'Cryptography' | openssl enc -aes128 -base64 -k 'testdata' -pbkdf2
U2FsdGVkX1/8sGvTYMz/1K8XPLJ4lw3rLDTrmT2U/fI=
root@kali:~# echo 'U2FsdGVkX1/8sGvTYMz/1K8XPLJ4lw3rLDTrmT2U/fI=' | openssl enc -d -aes128 -
base64 -k 'testdata' -pbkdf2
Cryptography
```

Data Encoding and Decoding





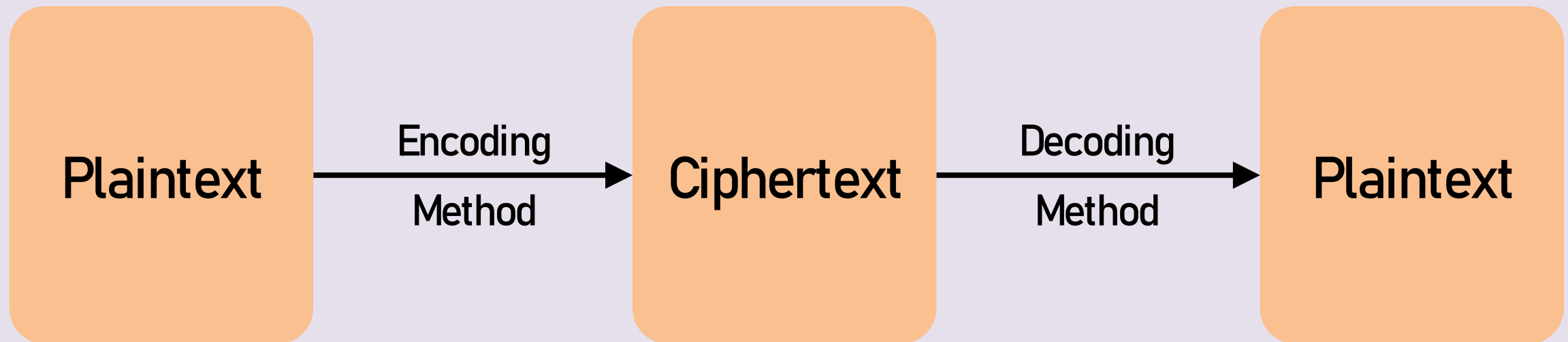
Overviews of Data Encoding and Decoding

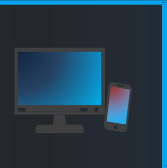
- **Data encoding** is the process of converting the data or a given sequence of **characters, symbols, alphabets** etc., into a **specified format**, for the secured data
- **Data decoding** is the reverse process of encoding which is to **extract the information** from the converted format to the **original format**

No key,
Two-way methods



Process of Data Encoding and Decoding





Encoding and Decoding Methods

Type	Details	Example of Methods
Numerical Encoding and Decoding	Represents numbers in different numeral systems	Binary, Octal, Decimal, Hexadecimal
Base-families Encoding and Decoding	Converts a binary data in Base-families format into text	Base32, Base45, Base58, Base62, Base64, Base85
Character Encoding and Decoding	Represents text using numerical values	ASCII, Unicode



Encoding and Decoding Methods (Cont)

Type	Details	Example of Methods
Symbolic Encoding and Decoding	Encodes characters using symbols instead of alphanumeric characters	Morse Code, QR Code, Braille, Emoji, Alien Alphabet
Text Obfuscation-based Encoding and Decoding	Modifies text to make it less readable while still reversible	ROT13, ROT47, Atbash, Rail Fence, Leetspeak, Zalgo
Text Hiding-based Encoding and Decoding	Concealing secret information within text	Tweet-secret, SpamMimic



Encoding and Decoding Methods (Cont)

Type	Details	Example of Methods
Esolang-based Encoding and Decoding	Encoding methods used in unconventional programming languages (Esolang: Esoteric Programming Language)	Brainfuck, JSFuck, Malbolge, Whitespace, Ook!
Slang-based Encoding and Decoding	Transforms text into informal, often used for secrecy, humor, or stylistic expression	Loo (Thai LGBTQ+), Khuk (Thai Prison), PAPAYA-Lang, Pig Latin



Numerical Encoding and Decoding



Binary

- **Binary** Encoding (**Base2**): represents data using two-symbol system **0s** and **1s**
- Commonly used in digital systems, computing, and networking
- Example:
A → **01000001**

Input

length: 12
lines: 1

+ folder share trash window

Bank Wongyos

Output

time: 0ms
length: 107
lines: 1

edit copy paste undo redo zoom

01000010 01100001 01101110 01101011 00100000 01010111
01101111 01101110 01100111 01111001 01101111 01110011



Binary (Cont)

- Python for binary encoding and decoding

```
def text_to_binary(text):  
    return ' '.join(format(ord(char), '08b') for char in text)  
def binary_to_text(binary):  
    chars = binary.split()  
    return ''.join(chr(int(b, 2)) for b in chars)  
  
binary = text_to_binary("Hi")  
print("Binary:", binary)  
text = binary_to_text(binary)  
print("Text:", text)
```



Octal

- **Octal** Encoding (**Base8**):
uses digits **0-7** to
represent binary data in
a compact form
- Used in computing, such
as **UNIX** file permissions
- Example:
Binary **1010** → Octal **12**
(**001010** → 12)

Input

length: 12
lines: 1

+    

Bank Wongyos

Output 

start: 46 time: 0ms
end: 46 length: 46
length: 0 lines: 1

102 141 156 153 40 127 157 156 147 171 157 163



Octal (Cont)

- Python for octal encoding and decoding

```
def text_to_octal(text):  
    return ' '.join(format(ord(c), 'o') for c in text)  
def octal_to_text(octal_str):  
    return ''.join(chr(int(o, 8)) for o in octal_str.split())  
  
print(octal_to_text("110 151")) # Output: Hi  
print(text_to_octal("Hi")) # Output: 110 151
```



Hexadecimal

- **Hexadecimal** Encoding (**Base16**): uses digits **0-9** and letters **A-F** to represent numbers
- Commonly used in computer **memory** addressing and **color codes**
- Example:
Binary **1010** → Hex **A**
Red (**255, 0, 0**) → **#FF0000**

Input	length: 12 lines: 1	+ Folder Share Trash Fullscreen
Bank Wongyos		
Output	time: 1ms length: 35 lines: 1	Save Copy Share Undo Zoom
42 61 6e 6b 20 57 6f 6e 67 79 6f 73		



Hexadecimal (Cont)

- Python for hexadecimal encoding and decoding

```
def text_to_hex(text):  
    return ' '.join(format(ord(char), '02x') for char in text)  
def hex_to_text(hex_string):  
    return ' '.join(chr(int(h, 16)) for h in hex_string.split())  
  
original_text = "RPCA Cyber Club"  
hex_encoded = text_to_hex(original_text)  
print("Hex Encoded:", hex_encoded)  
decoded_text = hex_to_text(hex_encoded)  
print("Decoded Text:", decoded_text)
```

Base-families Encoding and Decoding



Base32

- **Base32** Encoding: uses **32** characters, **A-Z** and **2-7** (avoids 0 and 1 to reduce confusion)
- **Each character** encodes **5 bits** of binary data
- **Padding** (=) is added when necessary
- Used in: QR codes, **DNS** encoding, Time-based **OTPs** (Google Authenticator)
- Example:
Input: "**Hello**"
Binary: **01001000 01100101 01101100 01101100 01101111**
Base32 Output: **JBSWY3DP**



Base32 (Cont)

Input

length: 12
lines: 1



Bank Wongyos

Output



time: 1ms
length: 24
lines: 1



IJQW42ZAK5XW4Z3ZN5ZQ====



Base32 (Cont)

- Python for Base32 encoding and decoding

```
import base64

def text_to_base32(text):
    return base64.b32encode(text.encode()).decode()
def base32_to_text(base32_str):
    return base64.b32decode(base32_str.encode()).decode()

original = "RPCA Cyber Club"
encoded = text_to_base32(original)
decoded = base32_to_text(encoded)
print("Original:", original)
print("Base32 Encoded:", encoded)
print("Decoded:", decoded)
```



Base45

- **Base45** Encoding: uses **45** characters
- Alphanumeric **A-Z, 0-9**, and **SPACE \$%*+-../:**
- Each character encodes **~5.5 bits** of binary data
- Used in: **QR codes** in EU COVID certificates
- Example:
Input: "**AB**"
Binary: **01000001 01000010**
Base45 Output: "**BB8**"



Base45 (Cont)

Input

length: 12
lines: 1



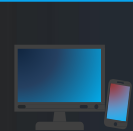
Bank Wongyos

Output

time: 1ms
length: 18
lines: 1



SH87/D:34-3ET3D14E



Base45 (Cont)

- Python for Base45 encoding and decoding

```
import base45

def text_to_base45(text):
    return base45.b45encode(text.encode()).decode()
def base45_to_text(b45_text):
    return base45.b45decode(b45_text.encode()).decode()

original = "RPCA Cyber Club"
encoded = text_to_base45(original)
decoded = base45_to_text(encoded)
print("Original Text:", original)
print("Base45 Encoded:", encoded)
print("Decoded Text:", decoded)
```



Base58

- **Base58** Encoding: uses **58** characters including **digits**, **upper case letters**, and **lower case letters** but removes **lookalike characters**, **0** (zero), **O** (capital letter O), **l** (small L), **I** (large i)
- Each character encodes **~5.86 bits**
- Used in: **Bitcoin** addresses, IPFS, Flickr short URLs
- Example:
Input: **"Hello"**
Base58 Output: **"9Ajdvzr"**



Base58 (Cont)

Input

length: 12
lines: 1



Bank Wongyos

Output

start: 15 time: 3ms
end: 15 length: 17
length: 0 lines: 1



2Fewnhy8RMB7i5oCE



Base58 (Cont)

■ Python for Base58 encoding and decoding

```
import base58

def text_to_base58(text):
    return base58.b58encode(text.encode()).decode()
def base58_to_text(base58_str):
    return base58.b58decode(base58_str.encode()).decode()

original = "RPCA Cyber Club"
encoded = text_to_base58(original)
decoded = base58_to_text(encoded)
print("Original:", original)
print("Base58 Encoded:", encoded)
print("Decoded:", decoded)
```



Base62

- **Base62** Encoding: uses **62** characters, **A-Z, a-z, 0-9**
- Each character encodes **~6 bits**
- Used in: **URL shortening** (e.g., YouTube links), Unique IDs
- Example:
Input: "**ChatGPT**"
Base62 Output: "**2MrXhX**"



Base62 (Cont)

Input

length: 12
lines: 1



Bank Wongyos

Output

time: 2ms
length: 16
lines: 1



QiWKhYn1CwVYm1tb



Base62 (Cont)

■ Python for Base62 encoding and decoding

```
import base62

def string_to_base62(text):
    return base62.encodebytes(text.encode())
def base62_to_string(encoded):
    return base62.decodebytes(encoded)

original = "RPCA Cyber Club"
encoded = string_to_base62(original)
decoded = base62_to_string(encoded)
print("Original:", original)
print("Base62 Encoded:", encoded)
print("Decoded:", decoded)
```



Base64

- **Base64** Encoding: uses **64** characters, **A-Z**, **a-z**, **0-9**, **+** **/**
- **Padding** (=) is added when necessary
- Each character encodes **6 bits**
- Used in: **Email** (MIME), **cryptography**, embedding **images** in HTML
- Example:
Input: "**Hello**"
Base64 Output: "**SGVsbG8=**"



Base64 (Cont)

Input

length: 12
lines: 1



Bank Wongyos

Output



time: 1ms
length: 16
lines: 1



QmFuayBXb25new9z



Base64 (Cont)

- Python for Base64 encoding and decoding

```
import base64

def text_to_base64(text):
    return base64.b64encode(text.encode()).decode()
def base64_to_text(b64_text):
    return base64.b64decode(b64_text.encode()).decode()

original = "RPCA Cyber Club"
encoded = text_to_base64(original)
decoded = base64_to_text(encoded)
print("Original Text:", original)
print("Base64 Encoded:", encoded)
print("Decoded Text:", decoded)
```




Base85

- **Base85** Encoding: uses **85** printable ASCII characters
- Each character encodes **~6.41 bits**, making it more space-efficient than Base64
- Used in: **Adobe** PostScript, **Ascii85**, binary-to-text encoding
- Example:
Input: "**Hello**"
Base85 Output: "**B0u!rD**"



Base85 (Cont)

Input

length: 12
lines: 1



Bank Wongyos

Output



time: 2ms
length: 15
lines: 1



6=FbI+BNK!B6eE:



Base85 (Cont)

- Python for Base85 encoding and decoding

```
import base64

def text_to_base85(text):
    return base64.b85encode(text.encode()).decode()
def base85_to_text(encoded_text):
    return base64.b85decode(encoded_text.encode()).decode()

original = "RPCA Cyber Club"
encoded = text_to_base85(original)
decoded = base85_to_text(encoded)
print("Original Text:", original)
print("Base85 Encoded:", encoded)
print("Decoded Text:", decoded)
```

Character Encoding and Decoding



ASCII and Unicode

- **ASCII** (American Standard Code for Information Interchange)
 - Map text characters to binary representations
 - Developed in the **1960s** for early computers
 - Uses **7-bit** encoding (values **0-127**)
 - Later **extended** to **8-bit** (Extended ASCII, values **0-255**)
- **Unicode**
 - Map text characters to binary representations (same as ASCII)
 - Developed to support **all characters** in **all languages**
 - Uses **variable-length** encoding (**UTF-8**, **UTF-16**, **UTF-32**)
 - Supports over **1.1 million** characters



ASCII and Unicode (Cont)

Dec	Hex	Oct	Char	Dec	Hex	Oct	Char	Dec	Hex	Oct	Char	Dec	Hex	Oct	Char
0	0	0		32	20	40	[space]	64	40	100	@	96	60	140	`
1	1	1		33	21	41	!	65	41	101	A	97	61	141	a
2	2	2		34	22	42	"	66	42	102	B	98	62	142	b
3	3	3		35	23	43	#	67	43	103	C	99	63	143	c
4	4	4		36	24	44	\$	68	44	104	D	100	64	144	d
5	5	5		37	25	45	%	69	45	105	E	101	65	145	e
6	6	6		38	26	46	&	70	46	106	F	102	66	146	f
7	7	7		39	27	47	'	71	47	107	G	103	67	147	g
8	8	10		40	28	50	(	72	48	110	H	104	68	150	h
9	9	11		41	29	51	)	73	49	111	I	105	69	151	i
10	A	12		42	2A	52	*	74	4A	112	J	106	6A	152	j
11	B	13		43	2B	53	+	75	4B	113	K	107	6B	153	k
12	C	14		44	2C	54	,	76	4C	114	L	108	6C	154	l
13	D	15		45	2D	55	-	77	4D	115	M	109	6D	155	m
14	E	16		46	2E	56	.	78	4E	116	N	110	6E	156	n
15	F	17		47	2F	57	/	79	4F	117	O	111	6F	157	o
16	10	20		48	30	60	0	80	50	120	P	112	70	160	p
17	11	21		49	31	61	1	81	51	121	Q	113	71	161	q
18	12	22		50	32	62	2	82	52	122	R	114	72	162	r
19	13	23		51	33	63	3	83	53	123	S	115	73	163	s
20	14	24		52	34	64	4	84	54	124	T	116	74	164	t
21	15	25		53	35	65	5	85	55	125	U	117	75	165	u
22	16	26		54	36	66	6	86	56	126	V	118	76	166	v
23	17	27		55	37	67	7	87	57	127	W	119	77	167	w
24	18	30		56	38	70	8	88	58	130	X	120	78	170	x
25	19	31		57	39	71	9	89	59	131	Y	121	79	171	y
26	1A	32		58	3A	72	:	90	5A	132	Z	122	7A	172	z
27	1B	33		59	3B	73	;	91	5B	133	[	123	7B	173	{
28	1C	34		60	3C	74	<	92	5C	134	\	124	7C	174	
29	1D	35		61	3D	75	=	93	5D	135	]	125	7D	175	}
30	1E	36		62	3E	76	>	94	5E	136	^	126	7E	176	~
31	1F	37		63	3F	77	?	95	5F	137	_	127	7F	177	

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
U+260x	☀	☂	☔	☄	☂	★	☆	☑	☒	☑	☑	☑	☑	☑	☑	☑
U+261x	☐	☑	☒	☓	☔	☕	☖	☗	☘	☙	☚	☛	☜	☝	☞	☟
U+262x	☠	☡	☢	☣	☤	☥	☦	☧	☨	☩	☪	☫	☬	☭	☮	☯
U+263x	☰	☱	☲	☳	☴	☵	☶	☷	☸	☹	☺	☻	☼	☽	☾	☿
U+264x	♀	♂	♂	♂	♂	♂	♂	♂	♂	♂	♂	♂	♂	♂	♂	♂
U+265x	♂	♂	♂	♂	♂	♂	♂	♂	♂	♂	♂	♂	♂	♂	♂	♂
U+266x	♠	♥	♦	♣	♠	♥	♦	♣	♠	♥	♫	♫	♫	♫	♫	♫
U+267x	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠
U+268x	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠
U+269x	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠
U+26Ax	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠
U+26Bx	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠
U+26Cx	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠
U+26Dx	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠
U+26Ex	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠
U+26Fx	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠	♠



ASCII and Unicode (Cont)

Input

length: 12
lines: 1

+    

Bank Wongyos

Output 

time: 0ms
length: 43
lines: 1

66 97 110 107 32 87 111 110 103 121 111 115

Charcode (CyberChef)



ASCII and Unicode (Cont)

- Python for ASCII encoding and decoding

```
def encode_ascii(text):  
    return text.encode('ascii')  
def decode_ascii(encoded):  
    return encoded.decode('ascii')  
  
text_ascii = "RPCA Cyber Club"  
encoded_ascii = encode_ascii(text_ascii)  
print("Encoded ASCII:", encoded_ascii)  
print("Decoded ASCII:", decode_ascii(encoded_ascii))
```




ASCII and Unicode (Cont)

- **Python** for **Unicode** encoding and decoding

```
def encode_utf8(text):  
    return text.encode('utf-8')  
def decode_utf8(encoded):  
    return encoded.decode('utf-8')  
  
text_ascii = "RPCA Cyber Club"  
encoded_utf8 = encode_utf8(text_unicode)  
print("Encoded UTF-8:", encoded_utf8)  
print("Decoded UTF-8:", decode_utf8(encoded_utf8))
```



Symbolic Encoding and Decoding



Morse Code

- **Morse Code** is a method of encoding textual information as a series of **dots (.)** and **dashes (-)**
- Each letter, number, or symbol is translated into a unique sequence of:
 - **.** (dot) for **short** signal
 - **-** (dash) for **long** signal
- Used in: telegraphy, aviation, amateur radio, and emergency signaling
- Example:
Input: **HELLO**
Morse: [**H** =] [**E** = .] [**L** = . - . .] [**L** = . - . .] [**O** = - - -]
Morse Code: - . . . - . . - - -



Morse Code (Cont)

	American	International
A	• —	• —
Ä		• — • —
Á		• — — • —
Å		• — — • —
B	— • • •	— • • •
C	• • •	— • — •
CH		— — — —
D	— • •	— • •
E	•	•
É		• • — • •
F	• — • •	• • — •
G	— — — •	— — — •
H	• • • •	• • • •
I	• •	• •
J	— • — — •	• — — — —
K	— • —	— • —
L	— — —	• — • •

	American	International
M	— — —	— — —
N	— •	— •
Ñ		— — • — —
O	• •	— — —
Ö		— — — •
P	• • • •	• — — •
Q	• • — •	— — • —
R	• • •	• — •
S	• • •	• • •
T	—	—
U	• • —	• • —
Ü		• • — —
V	• • • —	• • • —
W	• — — —	• — — —
X	• — • •	— • • —
Y	• • • •	— • — —
Z	• • • •	— — • •

	American	International
1	• — — • •	• — — — —
2	• • — • •	• • — — —
3	• • • — •	• • • — —
4	• • • • —	• • • • —
5	— — — —	• • • • •
6	• • • • •	— • • • •
7	— — — • •	— — — • •
8	— — • • •	— — — • •
9	— • • — —	— — — — •
0	— — — — —	— — — — —
@		• — — • — •

© Encyclopædia Britannica, Inc.



Morse Code (Cont)

Input

length: 12
lines: 1



Bank Wongyos

Output



start: 42 time: 0ms
end: 42 length: 42
length: 0 lines: 2



- - . - . - .
. - . - . - . - . - . - . - . - .



Morse Code (Cont)

■ Python for Morse Code encoding and decoding

```
MORSE_CODE_DICT = {  
    'A': '.-',      'B': '-...',   'C': '-.-.',   'D': '-..',  
    'E': '.',       'F': '..-.',   'G': '--.',    'H': '....',  
    'I': '..',       'J': '.---',   'K': '-.-',    'L': '-...',  
    'M': '--',       'N': '-.',     'O': '---',    'P': '-.-.',  
    'Q': '--.-',     'R': '.-.',    'S': '...',    'T': '-',  
    'U': '..-',       'V': '...-',   'W': '.--',    'X': '-.-.-',  
    'Y': '-.-',      'Z': '--..',   '0': '-----', '1': '.-----',  
    '2': '..---',     '3': '...--',  '4': '....-',   '5': '.....',  
    '6': '-.....',   '7': '--...',  '8': '---..',   '9': '----.',  
    ' ': '/',        ',': '--..--', '.': '.-.-.-', '?': '..-.-.',  
    '!': '-.-.-'     }  
}
```



Morse Code (Cont)

```
REVERSE_MORSE_DICT = {v: k for k, v in MORSE_CODE_DICT.items()}

def encode_morse(text):
    text = text.upper()
    return ' '.join(MORSE_CODE_DICT.get(char, '?') for char in text)

def decode_morse(morse_code):
    return ''.join(REVERSE_MORSE_DICT.get(code, '?') for code in
                    morse_code.split())
```



Morse Code (Cont)

```
# Example
plain_text = "HELLO WORLD"
encoded = encode_morse(plain_text)
decoded = decode_morse(encoded)

print("Plain Text:", plain_text)
print("Encoded Morse:", encoded)
print("Decoded Text:", decoded)
```




QR Code

- **QR Code** (Quick Response Code) is a type of **2D matrix** used to store and share data efficiently
- Shape: **Square matrix** of **black** and **white** modules (**pixels**)
- Capacity: Can store up to **7,089 numeric**, **4,296 alphanumeric**, or **2,953 bytes**
 - **Numeric** 0–9 only (highest compression)
 - **Alphanumeric** 0–9, A–Z, space, \$, %, *, +, -, ., /, :
 - **Byte** Supports any 8-bit character (UTF-8)
 - **Kanji** Double-byte character sets (Shift JIS)
- **QR codes** can encode: URLs, Plain text, Contact information (vCard), Wi-Fi credentials, GPS coordinates, Email, SMS templates



QR Code (Cont)

Input

length: 12
lines: 1

+ folder → trash list

Bank Wongyos

Output ✂ ✕

time: 6ms
length: 343
lines: 3

save copy share undo expand





QR Code (Cont)

- Python for QR code encoding and decoding

```
import qrcode

data = "https://rpca.ac.th"
qr = qrcode.make(data)
qr.save("qrcode.png")
```

```
from pyzbar.pyzbar import decode
from PIL import Image

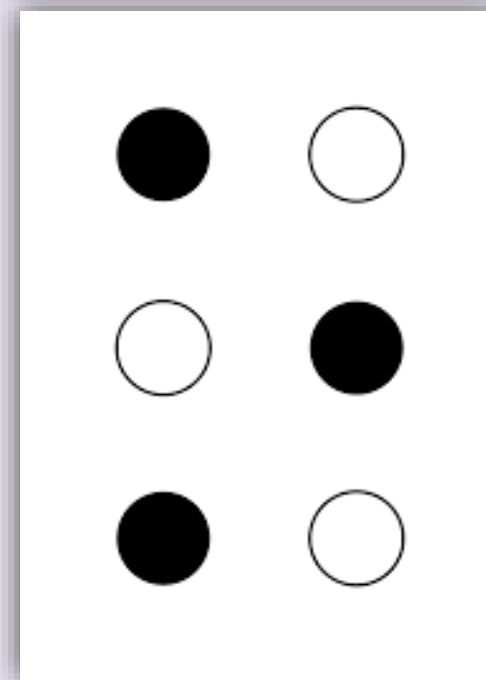
image = Image.open('qrcode.png')
decoded_objects = decode(image)

for obj in decoded_objects:
    print("Data:",
    obj.data.decode("utf-8"))
```



Braille

- **Braille** is a tactile writing system used by **visually impaired people**
- Each Braille character is made from up to **6 dots** in a **3×2 matrix**, encoded into **Unicode Braille Patterns**
- We often use **Unicode Braille Patterns** (U+2800 to U+28FF) to represent Braille **dots**
- Example:
Input: **BANK**
Braille Output: ⠠ ⠠ ⠠ ⠠



Braille Cell



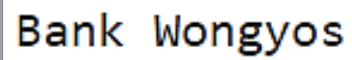
Braille (Cont)

a	b	c	d	e	f	g	h	i	j	1	2
k	l	m	n	o	p	q	r	s	t	3	4
u	v	x	y	z	ç	é	à	è	ù	5	6
â	ê	î	ô	û	ë	ï	ü	œ	w	7	8
,	;	:	.	?	!	()	«	*	»	9	0



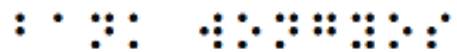
Input

```
length: 12
lines: 1
```



Output

```
time: 1ms
length: 12
lines: 1
```





Braille (Cont)

- Python for Braille encoding and decoding

```
BRaille_DICT = {
    'A': '⠁', 'B': '⠃', 'C': '⠉', 'D': '⠙', 'E': '⠑',
    'F': '⠋', 'G': '⠗', 'H': '⠨', 'I': '⠊', 'J': '⠚',
    'K': '⠓', 'L': '⠕', 'M': '⠎', 'N': '⠞', 'O': '⠥',
    'P': '⠖', 'Q': '⠘', 'R': '⠞', 'S': '⠠', 'T': '⠟',
    'U': '⠥', 'V': '⠦', 'W': '⠢', 'X': '⠭', 'Y': '⠮', 'Z': '⠵',
    ' ': ' ', # space
    '0': '⠼', '1': '⠁', '2': '⠃', '3': '⠉', '4': '⠙',
    '5': '⠑', '6': '⠋', '7': '⠗', '8': '⠨', '9': '⠊'
}
```



Braille (Cont)

```
REVERSE_BRAILLE_DICT = {v: k for k, v in BRAILLE_DICT.items()}

def encode_braille(text):
    return ''.join(BRAILLE_DICT.get(c.upper(), '?') for c in text)
def decode_braille(braille):
    return ''.join(REVERSE_BRAILLE_DICT.get(c, '?') for c in braille)

plain = "RPCA Cyber Club"
braille = encode_braille(plain)
decoded = decode_braille(braille)
print("Original:", plain)
print("Braille Encoded:", braille)
print("Braille Decoded:", decoded)
```




Emoji

- **Emoji** is a form of **symbolic** for representing **letters**, **numbers**, or **binary** using emojis
- Used in: Text obfuscation, Game design / puzzles, and Social media & messaging
- Example:
Input: **"HELLO"**
Encoded Output: 🏠 🐘 🦁 🦁 🍷



Emoji (Cont)

BASE¹⁰⁰ CIPHER

★ PLAINTEXT TO ENCODE WITH EMOJIS ?

Bank Wongyos

▶ ENCODE

Results



Bank Wongyos



Base100 ¹⁰⁰ - dCode



Emoji (Cont)

■ Python for Emoji encoding and decoding

```
# Emoji mappings (you can customize this dictionary)
EMOJI_DICT = {
    'A': '🍎', 'B': '🐝', 'C': '🌩️', 'D': '🐶', 'E': '🐘',
    'F': '🐸', 'G': '🦒', 'H': '🏠', 'I': '🍦', 'J': '👤',
    'K': '🔧', 'L': '🐼', 'M': '🌍', 'N': '🏢', 'O': '👱',
    'P': '🍷', 'Q': '🦁', 'R': '🌈', 'S': '👉', 'T': '🌴',
    'U': '☂️', 'V': '🚫', 'W': '🌩️', 'X': '✖️', 'Y': '🏰', 'Z': '⚡',
    ' ': '□'
}
```



Emoji (Cont)

```
# Reverse mapping for decoding
REVERSE_EMOJI_DICT = {v: k for k, v in EMOJI_DICT.items()}

# Encoding function
def encode_emoji(text):
    return ''.join(EMOJI_DICT.get(c.upper(), '?') for c in text)

# Decoding function
def decode_emoji(emoji_text):
    return ''.join(REVERSE_EMOJI_DICT.get(c, '?') for c in emoji_text)
```



Emoji (Cont)

```
plain = "RPCA Cyber Club"
emoji_encoded = encode_emoji(plain)
emoji_decoded = decode_emoji(emoji_encoded)

print("Text:", plain)
print("Emoji Encoded:", emoji_encoded)
print("Emoji Decoded:", emoji_decoded)
```



- Input: "HELLO WORLD 2025"

Chapter 6



Alien Alphabet (Cont)

ALIEN LANGUAGE ENCODER

★ ALIEN LANGUAGE PLAINTEXT ?

Bank Wongyos

★ SHOW AS ☒ UNICODE CHARACTERS (𐄧𐄫𐄷𐄫) (EASY COPY/PASTE)

☐ IMAGES (AS ABOVE)

▶ ENCRYPT

Results

BANK WONGYOS

𐄧𐄫𐄷𐄫 𐄧𐄫𐄷𐄫𐄷𐄫𐄷𐄫𐄷𐄫

Alien Language 𐄧𐄫𐄷𐄫 - dCode

Tag(s) : Symbol Substitution



Alien Alphabet (Cont)

- Python for Alien Alphabet encoding and decoding

```
ALIEN_ALPHABET = {  
    'A': 'Δ', 'B': 'β', 'C': 'ç', 'D': 'Ð', 'E': 'ξ',  
    'F': 'Ƒ', 'G': 'Ƴ', 'H': '⌘', 'I': 'Ꞥ', 'J': 'ĵ',  
    'K': 'Ƙ', 'L': 'ɔ', 'M': '♏', 'N': 'ℵ', 'O': '⊙',  
    'P': 'φ', 'Q': 'ℚ', 'R': '₹', 'S': '℥', 'T': '†',  
    'U': 'ℰ', 'V': '√', 'W': 'ω', 'X': 'ℵ', 'Y': 'ɥ', 'Z': '℥',  
    ' ': ' ',  
    '0': '⓪', '1': '①', '2': '②', '3': '③', '4': '④',  
    '5': '⑤', '6': '⑥', '7': '⑦', '8': '⑧', '9': '⑨'  
}
```




Alien Alphabet (Cont)

```
# Reverse mapping for decoding
REVERSE_ALIEN_ALPHABET = {v: k for k, v in ALIEN_ALPHABET.items()}

def encode_alien(text):
    return ''.join(ALIEN_ALPHABET.get(c.upper(), '?') for c in text)

def decode_alien(encoded):
    return ''.join(REVERSE_ALIEN_ALPHABET.get(c, '?') for c in encoded)
```



Alien Alphabet (Cont)

```
plain = "RPCA Cyber Club"  
alien_encoded = encode_alien(plain)  
alien_decoded = decode_alien(emoji_encoded)  
  
print("Text:", plain)  
print("Alien Encoded:", alien_encoded)  
print("Alien Decoded:", alien_decoded)
```



Text Obfuscation-based Encoding and Decoding

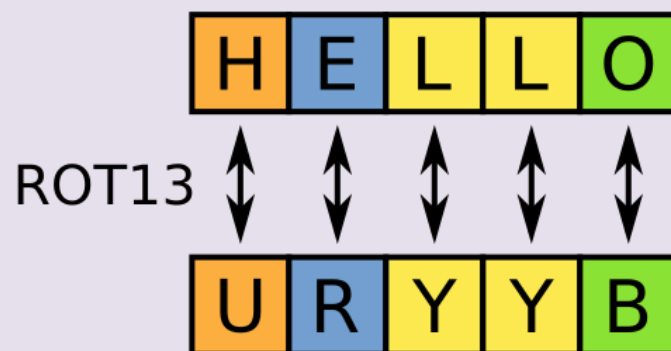
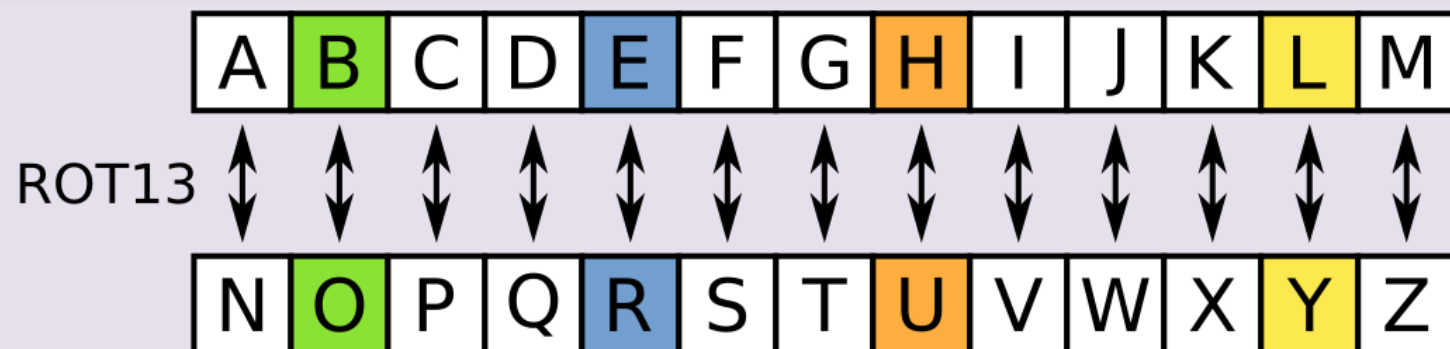


ROT13

- **ROT13** (Rotate by 13 places) is a simple **substitution cipher** used to encode and decode text by **shifting** each letter of the alphabet **13 positions** forward
- For each **letter**:
 - If the letter is in **A–M/a–m**, shift it **forward by 13**
 - If the letter is in **N–Z/n–z**, shift it **backward by 13**
- **Non-alphabetic characters** (e.g., digits, punctuation) are left **unchanged**
- Example:
Input: **"Hello, World!"**
ROT13 Output: **Uryyb, Jbeyq!**



ROT13 (Cont)





ROT13 (Cont)

Input

length: 12
lines: 1



Bank Wongyos

Output

time: 1ms
length: 12
lines: 1



Onax Jbatlbf



ROT13 (Cont)

■ Python for ROT13 encoding and decoding

```
def rot13(text):  
    result = []  
    for char in text:  
        if 'A' <= char <= 'Z':  
            result.append(chr((ord(char) - ord('A') + 13) % 26 + ord('A')))  
        elif 'a' <= char <= 'z':  
            result.append(chr((ord(char) - ord('a') + 13) % 26 + ord('a')))  
        else:  
            result.append(char)  
    return ''.join(result)  
  
encoded = rot13('Hello')  
decoded = rot13(encoded)
```



ROT47

- **ROT47** is an extended form of ROT13 that works on the printable ASCII character **range (33–126)**
- It rotates each character **47 positions** forward, wrapping around if needed
- Works on characters from **! (ASCII 33)** to **~ (ASCII 126)**
- For each **character**:
 - If the **ASCII** value is in the **range 33–126**, rotate **47 positions**
 - Otherwise, leave it **unchanged** (e.g., newline, tab, Unicode)



ROT47 (Cont)

Input

length: 12
lines: 1



Bank Wongyos

Output

time: 0ms
length: 12
lines: 1



q2?< (@?8J@D



ROT47 (Cont)

■ Python for ROT47 encoding and decoding

```
def rot47(text):  
    result = []  
    for char in text:  
        ascii_code = ord(char)  
        if 33 <= ascii_code <= 126:  
            result.append(chr(33 + ((ascii_code - 33 + 47) % 94)))  
        else:  
            result.append(char)  
    return ''.join(result)
```

```
encoded = rot47('Hello, World!')      # Output: "w6==@ (@C=5D"  
decoded = rot47(encoded)               # Output: "Hello, World!"
```



Atbash

- **Atbash** is a monoalphabetic substitution cipher originally used for the **Hebrew alphabet**, it maps each letter to its **opposite** in the alphabet: $A \leftrightarrow Z$, $B \leftrightarrow Y$, $C \leftrightarrow X \dots M \leftrightarrow N$
- For each alphabetic character:
 - If **uppercase**: $\text{cipher_char} = \text{chr}(\text{ord}('Z') - (\text{ord}(\text{char}) - \text{ord}('A')))$
 - If **lowercase**: $\text{cipher_char} = \text{chr}(\text{ord}('z') - (\text{ord}(\text{char}) - \text{ord}('a')))$
 - **Non-alphabetic characters** (digits, symbols, spaces) are left **unchanged**
- Example:
Input: **"HELLO world!"**
Output: **"SV00L dliow!"**



Atbash (Cont)

Input

length: 12
lines: 1



Bank Wongyos

Output

time: 1ms
length: 12
lines: 1



Yzmp Dlmtblh



Atbash (Cont)

- Python for Atbash encoding and decoding

```
def atbash_cipher(text):  
    result = []  
    for char in text:  
        if char.isupper():  
            result.append(chr(ord('Z') - (ord(char) - ord('A'))))  
        elif char.islower():  
            result.append(chr(ord('z') - (ord(char) - ord('a'))))  
        else:  
            result.append(char)  
    return ''.join(result)  
  
output = atbash_cipher("HELLO world!")  
original = atbash_cipher(output)
```



Rail Fence

- **Rail Fence** is a classic **transposition cipher** that writes the plaintext in a **zigzag pattern** (like a fence) across **multiple "rails" (rows)**, then reads off the ciphertext row-by-row
- Steps to **Encode**:
 - Choose a **number of rails** (e.g., 3)
 - Write the **plaintext** in a **zigzag pattern** across these rails
 - Read the **text row by row** to get the ciphertext
- Steps to **Decode**:
 - Determine the **zigzag pattern** based on the number of rails and message length
 - **Place characters** accordingly
 - **Read off row-by-row** in a zigzag to retrieve the original message



Rail Fence (Cont)

- Plaintext:

WEAREDISCOVEREDFLEEATONCE

- Writing Zigzag (Rails 3):

W . . . E . . . C . . . R . . . L . . . T . . . E
. E . R . D . S . O . E . E . F . E . A . O . C .
. . A . . . I . . . V . . . D . . . N . . . E . .

- Read Row by Row):

WECRL TEERD SOEEF EAOCA IVDEN

- Ciphertext:

WECRLTEERDSOEEFEAOCAIVDEN



Rail Fence (Cont)

Input

length: 12
lines: 1



Bank Wongyos

Output

time: 0ms
length: 12
lines: 1



Bn ogoakWnys



Rail Fence (Cont)

■ Python for Rail Fence encoding and decoding

```
def rail_fence_encode(text, rails):
    fence = [[] for _ in range(rails)]
    rail = 0
    direction = 1
    for char in text:
        fence[rail].append(char)
        rail += direction
        if rail == 0 or rail == rails - 1:
            direction *= -1
    return ''.join(''.join(row) for row in fence)

print(rail_fence_encode("WEAREDISCOVEREDFLEEATONCE", 3))
# Output: WECRLTEERDSOEEFEAOCAIVDEN
```



Rail Fence (Cont)

```
def rail_fence_decode(cipher, rails):  
    n = len(cipher)  
    pattern = [['\n' for _ in range(n)] for _ in range(rails)]  
    index, down = 0, None  
    row, col = 0, 0  
  
    # Step 1: Create pattern  
    for i in range(n):  
        if row == 0:  
            down = True  
        if row == rails - 1:  
            down = False  
        pattern[row][col] = '*'  
        col += 1  
        row += 1 if down else -1
```



Rail Fence (Cont)

```
# Step 2: Fill in pattern with cipher
idx = 0
for i in range(rails):
    for j in range(n):
        if pattern[i][j] == '*' and idx < n:
            pattern[i][j] = cipher[idx]
            idx += 1

# Step 3: Read the message
result = []
row, col = 0, 0
for i in range(n):
    if row == 0:
        down = True
    #...
```



Rail Fence (Cont)

```
#...
if row == rails - 1:
    down = False
result.append(pattern[row][col])
col += 1
row += 1 if down else -1

return ''.join(result)

print(rail_fence_decode("WECRLTEERDSOEEFEAOCAIVDEN", 3))
# Output: WEAREDISCOVEREDFLEEATONCE
```



Leetspeak

- **Leetspeak** (or **1337speak**) is a form of **textual obfuscation** where letters are replaced with **similar-looking** numbers or symbols
- **"Leet"** is derived from **"elite"**, and **"1337"** means leet
- Used in: **Hacker culture**, Bypassing filters, Online gaming, Programming fun
- Example:
Input: **"Leet Speak"**
Output: **"1337 5|*34|<"**



Leetspeak (Cont)

WRITE IN LEET SPEAK

★ LEET PLAIN TEXT (?)

Bank Wongyos

★ MODE ☒ SIMPLE: 51MP13

Results

(1) Basique Leet Writing
(2) Complex Leet Writing

84NK W0N6Y05

Leet Speak 1337 - [dCode](#)



Leetspeak (Cont)

■ Python for Leetspeak encoding and decoding

```
LEET_DICT = {
    'A': '4', 'B': '8', 'C': '<', 'D': '|)', 'E': '3', 'F': '|=', 'G': '6',
    'H': '#', 'I': '1', 'J': '_|', 'K': '|<', 'L': '1', 'M': '/\\//\\',
    'N': '|\\|', 'O': '0', 'P': '|*', 'Q': '0,', 'R': '|2', 'S': '5',
    'T': '7', 'U': '|_|', 'V': '\\//', 'W': '\\//\\//', 'X': '><', 'Y': '`/',
    'Z': '2'
}

def leetspeak_encode(text):
    return ''.join(LEET_DICT.get(c.upper(), c) for c in text)

text = "Leet Speak"
leetspeak_encode(text)  # Output: "1337 5|*34|<"
```



Zalgo

- Input:** "Zalgo is coming"
Output: "ᲞᲠᲣ ᲗᲢᲘᲚ ᲙᲟᲑᲓᲡᲐ"



Zalgo (Cont)

ZALGO WRITER (DIACRITICS OBFUSCATOR)

★ ZALGO MESSAGE TO HIDE (?)

Bank Wongyos

Results

Bank Wongyos

Baṅk-Wōŋgyōs

Zalgo Writing - dCode



Zalgo (Cont)

- Python for Zalgo encoding and decoding

```
import unicodedata
import random

# Unicode combining marks
zalgo_up = [chr(c) for c in range(0x0300, 0x036F + 1)]
zalgo_down = [chr(c) for c in range(0x0316, 0x031B + 1)]
zalgo_mid = [chr(c) for c in range(0x0334, 0x0338 + 1)]

def zalgo(text, intensity=1):
    result = ""
    for char in text:
        if char.isalpha():
            result += char
```



Zalgo (Cont)

```
        result += ''.join(random.choices(zalgo_up, k=random.randint(1, 3) * intensity))
        result += ''.join(random.choices(zalgo_down, k=random.randint(1, 3) * intensity))
        result += ''.join(random.choices(zalgo_mid, k=random.randint(0, 2) * intensity))
    else:
        result += char
    return result

def remove_zalgo(text):
    return ''.join(c for c in unicodedata.normalize('NFKD', text) if not
unicodedata.combining(c))

text = "Zalgo is coming"
zalgo(text)
```



Text Hiding-based Encoding and Decoding



Tweet-secret

- **Tweet-secret (steganography)** is a method of **hiding messages** in **tweets** or other **short-form texts** (e.g., Twitter, Threads, Mastodon)
- Tweet-secret hides the existence of the message, making it appear like a normal, innocent post
- Some **popular techniques** used for **tweet/text steganography**
 - Uses **invisible Unicode characters** (e.g., Zero Width Space, Zero Width Non-Joiner)
 - Replaces standard characters with visually **identical Unicode characters** from **other languages/scripts**
 - Uses **capitalization** or **punctuation** (commas, periods, exclamations)
 - Use **spaces** and **tabs** at the end of lines to encode binary



Tweet-secret (Cont)

Encode

Tweet (allows Hidden Message length 34)

Tweet-secret (steganography) is a method of hiding messages in tweets or other short-form texts (e.g., Twitter, Threads, Mastodon)

Hidden Message (length: 13)

bank wongyos



Tweet to post (length: 131)

Tweet-secret (steganography) is a method of hiding messages in tweets or other short-form texts (e.g.,

Decode

Tweet

Tweet-secret (steganography) is a method of hiding messages in tweets or other short-form texts (e.g., Twitter, Threads, Mastodon)

Hidden Message

bank wongyos

<https://wulfsige.com/crypto/>



Tweet-secret (Cont)

Encode

Public message (allows Hidden Message length 41)

Tweet-secret (steganography) is a method of hiding messages in tweets or other short-form texts (e.g., Twitter, Threads, Mastodon)

Hidden Message (length: 13)

bank wongyos



Secret Message to send (length: 141) (uneditable / read only)

Tweet-secret (steganography) is a method of hiding messages in tweets or other short-form texts (e.g., Twitter, Threads, Mastodon)

Decode

Potentially Secret Message.

Tweet-secret (steganography) is a method of hiding messages in tweets or other short-form texts (e.g., Twitter, Threads, Mastodon)

Hidden Message (uneditable / read only)

bank wongyos

<https://holloway.nz/steg/>



SpamMimic

- SpamMimic is a text steganography technique that hides secret messages inside texts that look like spam emails
- Developed by Peter Wayner (author of Disappearing Cryptography)
- Encode plaintext messages into spam-like text, making them seem irrelevant to humans but carrying encoded data that can be decoded back into the original message
- Converts plaintext into base-5 encoding and translates that encoding into a sequence of "spammy" phrases using a fixed grammar
- Produces output like: *"Dear Friend, I am a Nigerian prince who has a great deal for you. Send money..."*



SpamMimic (Cont)



Encode

Enter your short secret message:

<https://spammimic.com/>



Encoded

Your message **Bank Wongyos** gets encoded into spam as:

Dear Friend , We know you are interested in receiving red-hot news . If you no longer wish to receive our publications simply reply with a Subject: of "REMOVE" and you will immediately be removed from our club ! This mail is being sent in compliance with Senate bill 2416 , Title 6 ; Section 302 . THIS IS NOT A GET RICH SCHEME . Why work for somebody else when you can become rich within 86 weeks ! Have you ever noticed people love convenience and nearly every commercial on television



Esolang-based Encoding and Decoding



Brainfuck

- **Brainfuck** is a **minimalist esolang** (esoteric programming language) designed to challenge and amuse programmers
- Created by **Urban Müller** in 1993, Brainfuck uses only **eight commands** and operates on an **array of memory** cells, making it extremely low-level, yet Turing-complete
- It's called "**Brainfuck**" because writing or understanding code in it is... **mentally intense**
- Example of "**Hello World!**" →
+++++ [>+++++>+++++>+++>+<<<<-
>+ .>+ .+++++ . .+++ .>+ .<<+++++ .> .+++ .----- .--
----- .>+ .> .



Brainfuck (Cont)

- Brainfuck **Syntax** and **Commands**

Command	Meaning
>	Move pointer to the right
<	Move pointer to the left
+	Increment the byte at the pointer
-	Decrement the byte at the pointer
.	Output the byte at the pointer (ASCII)
,	Input a byte and store at pointer
[	Jump past matching] if current byte is zero
]	Jump back to matching [if current byte is nonzero



Brainfuck (Cont)

BRAINFUCK ENCODER

★ PLAINTEXT TO CODE IN BRAINF**K (?)

Bank Wongyos



★ ADD A SEPARATOR BETWEEN INSTRUCTIONS ☐

<https://www.dcode.fr/brainfuck-language>

Results



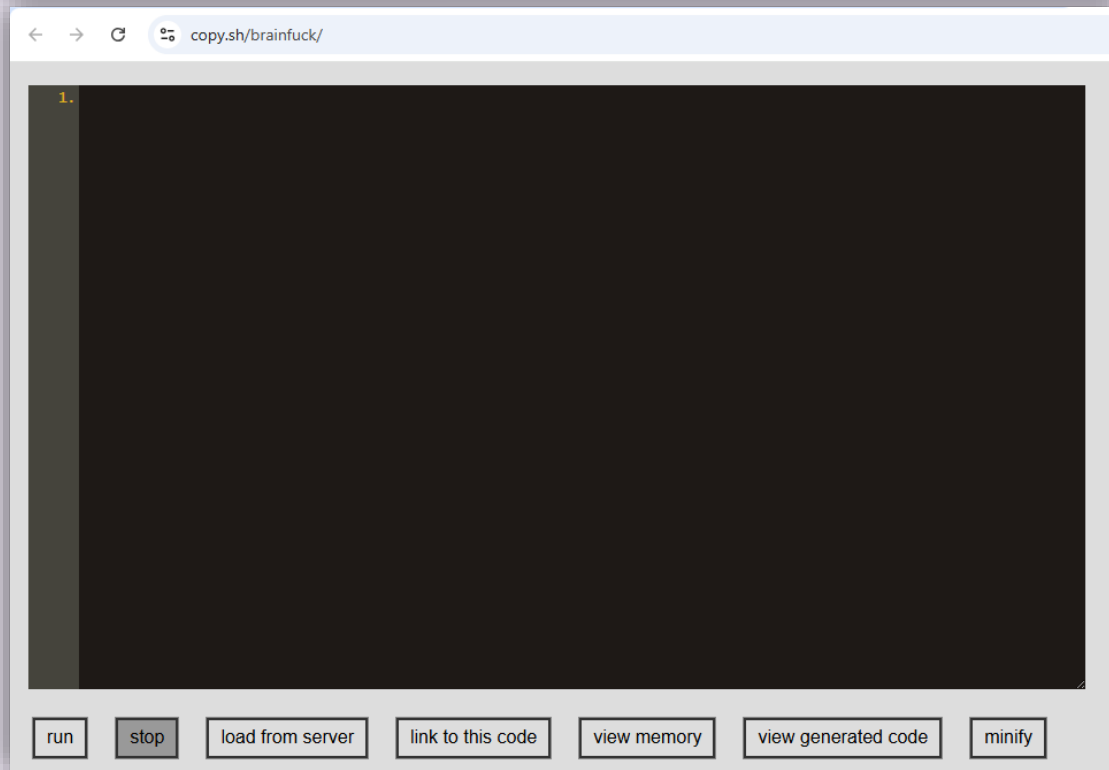
Bank Wongyos

```
+++++++[>+>+++>+++++++>+++++++  
<<<<-]>>>-----.>---.+++++++-----.<<+>.>>-  
-----  
- .+++++++-----  
- .+++++++----- .++++.
```

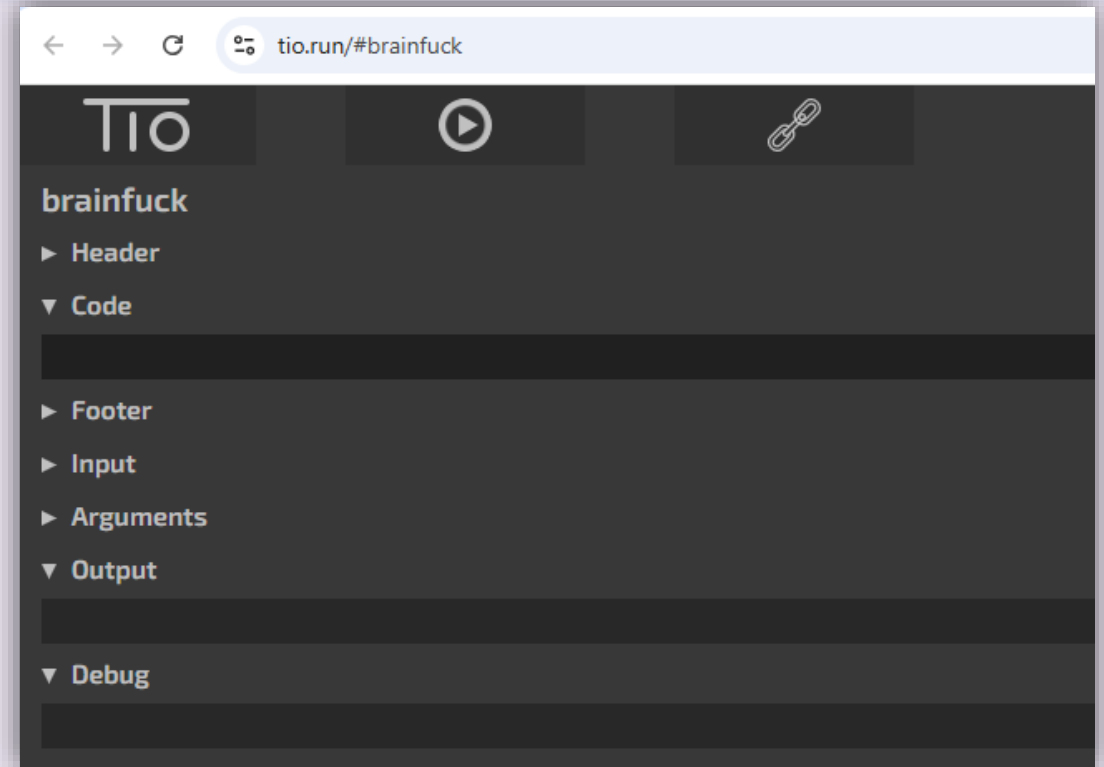
Brainfuck - [dCode](#)



Brainfuck (Cont)



<https://copy.sh/brainfuck/>

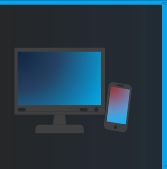


<https://tio.run/#brainfuck>



JSFuck

- JSFuck is an **esoteric** and **obfuscated** subset of **JavaScript** that uses only **six** characters **[] () ! +**
- JSFuck constructs numbers, strings, functions, and objects step-by-step using **expressions** like:
 - **`+[] → 0`**
 - **`![] → false`**
 - **`!![] → true`**
 - **`([]+[]) → ""`**
 - **`([]+[])[constructor] → "function String() { [native code] }"`**
- Then JSFuck uses these to **access characters** (**`["c"]`**, **`["o"]`**, etc.), **build strings**, and even **call functions** like **`alert`**



JSFuck (Cont)

- Some **correspondences** of JSFuck

JSFuck	Equivalent JavaScript
<code>! []</code>	<code>false</code>
<code>!! []</code>	<code>true</code>
<code>+ []</code>	<code>0</code>
<code>+!+ []</code>	<code>1</code>
<code>!+ []+!+ []</code>	<code>2</code>
<code>[+!+ []]+[+ []]</code>	<code>10</code>
<code>(! []+ ([]+ [])) [+!+ []]</code>	<code>a</code>





JSFuck (Cont)

JSFuck DECODER

★ CODE (ONE LINE) OR TEXT ?

Bank wongyos

★ ENCODING ☒ CODE AS TEXT WITH A POPUP ALERT()
☐ CODE AS JAVASCRIPT COMMAND (NO CHANGE)

▶ ENCODE

<https://www.dcode.fr/jsfuck-language>

Results



```
[][(![]+[]) [+[]]+(![]+[]) [!+[]+!+[]]+(![]+[])  
[+!+[]]+(![]+[]) [+[]]] [([]) [(![]+[]) [+[]]+(!  
[]+[]) [!+[]+!+[]]+(![]+[]) [+!+[]]+(![]+[]) [+  
[]]+[]) [!+[]+!+[]]+!+[]]+(![]+[]) [(![]+[]) [+  
[]]+(![]+[]) [!+[]+!+[]]+(![]+[]) [+!+[]]+(!  
[]+[]) [+[]]] [+!+[]+[]]+([[] [[]]+[]) [+!+  
[]]+(![]+[]) [!+[]+!+[]]+!+[]]+(![]+[]) [+[]]+  
(!![]+[]) [+!+[]]+([[] [[]]+[]) [+[]]+([[] [(![]+  
[])+[]]+(![]+[]) [!+[]+!+[]]+(![]+[]) [+!+[]]+
```



JSFuck (Cont)

jsfuck.com

JSFuck

JSFuck is an esoteric and educational programming style based on the atomic parts of JavaScript. It uses only six different characters to write and execute code.

It does not depend on a browser, so you can even run it on Node.js.

Use the form below to convert your own script. Uncheck "eval source" to get back a plain string.

☒ Eval Source ☒ Run In Parent Scope

```
[ ][( ! ) + [ ] ) + ! + [ ] + ( ! [ ] + [ ] ) + [ ] [ ( [ ] [ ( ! [ ] + [ ] ) + ! + [ ] + ( ! [ ] + [ ] ) + [ ] ] + [ ] )  
[ ! + [ ] + ! + [ ] + ! + [ ] + ( ! [ ] + [ ] [ ( ! [ ] + [ ] ) + ! + [ ] + ( ! [ ] + [ ] ) + [ ] ] + [ ] + [ ] + [ ] + [ ] ) + [ ]  
[ ] + [ ] + [ ] + ! + [ ] + ( ! [ ] + [ ] ) + ! + [ ] + ! + [ ] + ! + [ ] + ( ! [ ] + [ ] ) + [ ] + [ ] + ( ! [ ] + [ ] ) + ! + [ ] +  
( [ ] [ ] + [ ] ) + [ ] + ( [ ] [ ( ! [ ] + [ ] ) + ! + [ ] + ( ! [ ] + [ ] ) + [ ] ] + [ ] ) + ! + [ ] + ! + [ ] + ! + [ ] +
```

1756 chars [Run This](#)

<https://jsfuck.com/>

github.com/aemkei/jsfuck

README WTFPL license

JSFuck [] () ! +

JSFuck is an esoteric and educational programming style based on the atomic parts of JavaScript. It uses only six different characters to write and execute code.

It does not depend on a browser, so you can even run it on Node.js.

Demo: jsfuck.com

By [@aemkei](#) and [friends](#).

Example

The following source will do an `alert(1)` :

```
[ ][( ! ) + [ ] ) + ! + [ ] + ( ! [ ] + [ ] ) + [ ] [ ( [ ] [ ( ! [ ] + [ ] ) + ! + [ ] + ( ! [ ] + [ ] ) + [ ] ] + [ ] )  
[ ! + [ ] + ! + [ ] + ! + [ ] + ( ! [ ] + [ ] [ ( ! [ ] + [ ] ) + ! + [ ] + ( ! [ ] + [ ] ) + [ ] ] + [ ] + [ ] + [ ] + [ ] ) + [ ]  
[ ] + [ ] + [ ] + ! + [ ] + ( ! [ ] + [ ] ) + ! + [ ] + ! + [ ] + ! + [ ] + ( ! [ ] + [ ] ) + [ ] + [ ] + ( ! [ ] + [ ] ) + ! + [ ] +  
( [ ] [ ] + [ ] ) + [ ] + ( [ ] [ ( ! [ ] + [ ] ) + ! + [ ] + ( ! [ ] + [ ] ) + [ ] ] + [ ] ) + ! + [ ] + ! + [ ] + ! + [ ] +
```

<https://github.com/aemkei/jsfuck>



Malbolge

- **Malbolge** (/mæl'boʊldʒ/) is one of the most difficult **esoteric programming languages** ever created, it designed by **Ben Olmstead** in 1998
- The name comes from "**Malebolge**", the **eighth circle of hell** in **Dante's *Inferno***, a fitting metaphor for its complexity
- **Malbolge instructions** are derived from a combination of the **character** at that **memory location**, its **position**, each character must be **encoded specially**, and memory values mutate over time, and for the decoding step uses the **cryptic transformation table**
- Example of "**Hello World**":

```
D' ` ; : L" ~ Z } Y3VyCvu@csqM_ : n1*6"EDC1e/bx`+_ ) : rwpunm3kSinmlkdib (IHdc\"!Y^] \ [ZYXQuU  
TMRQJImG/KJIHG@d>&< ; : 9>=6Z498165.-Q10/.' &J*) (!g}C0
```



Malbolge (Cont)

Desired output:

You can use these special characters:

<code>\p [char]</code>	Display prompt, assume user inputs char
<code>\x</code>	Erase user input
<code>\s</code>	Don't erase user input till next output
<code>\\</code>	Display \

Bank Wongyos



Generate

Resume from file

```
D'`$@L\!IZ|j2DTBu-?b=No-J\I6FEWVUT".a,{)
(xwYon4UTjih.lkjiHJI_%FE[!BXwV[ZSwWPUTSRKJnNM/KDhHA@dDCB$@9]7<5Y3y705.-Q10/.-
&+*#G4
```

Length: **124**

Generation complete.

<https://zb3.me/malbolge-tools/>

Malbolge Tools

Interpreter

Generator

Load program from file

Choose example:

Enter for zb3!



Load

Program code: ☐ Normalized

```
D'`$@L\!IZ|j2DTBu-?b=No-J\I6FEWVUT".a,{)
(xwYon4UTjih.lkjiHJI_%FE[!BXwV[ZSwWPUTSRKJnNM/KDhHA@dDCB$@9]7<5Y3y705.-Q10/.-
&+*#G4
```



Execute

Bank Wongyos

Program finished.



Malbolge (Cont)

Malbolge

Malbolge, invented by Ben Olmstead in 1998, is an esoteric programming language designed to be as difficult to program in as possible. And so it is :-)

Hello World ▾

Library

About the project

New program

Program code:

```
1 ('&%:9]!~}|z2Vxwv-,POqponl$Hjig%eB@@>}=<M:9wv6WsU2T|nm-,jcl(I&%$#"
2 `CB]V?Tx<uVt`~Rpo3NlF.Jh++FdbCBA@?]|~|4XzyTT43Qsqq(Lnmkj"Fhg${z@>
3
4
5
6
7
8
9
10
```

size: 132 lines: 2

More

Run program

<https://malbolge.doleczek.pl/>



Whitespace

- **Whitespace** is an **esoteric** programming language created in 2003 by **Edwin Brady** and **Chris Morris**, in which only **whitespace characters** (spaces, tabs, and **linefeeds**) are considered meaningful
- All non-whitespace characters are ignored, making the code **invisible** to the eye in most editors
- Whitespace uses three characters:
 - **Space** (): 0
 - **Tab** (→): 1
 - **Linefeed** (↵): Instruction separator



Whitespace (Cont)

CODE SOME TEXT WITH WHITESPACE

★ WHITESPACE (ASCII) PLAINTEXT ?

Bank wongyos



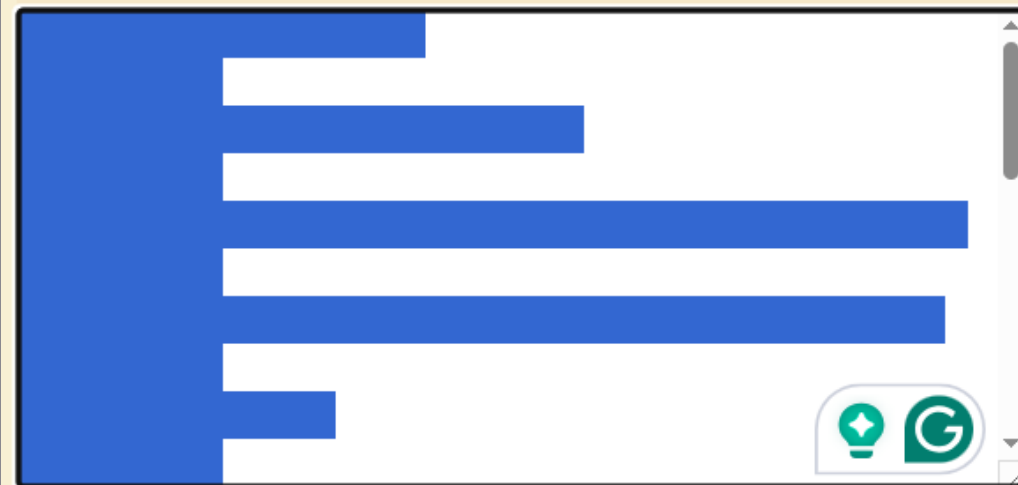
★ DISPLAY STL: S=SPACE, T=TAB, L=LINE FEED ☐

▶ ENCRYPT

Results



Remainder: encoded whitespace messages are not visible



Whitespace Language - [dCode](#)



Whitespace (Cont)

■ Python for Whitespace encoding and decoding

```
def hide_in_whitespace(text):
    binary = ''.join(f"{ord(c):08b}" for c in text)
    return ''.join(' ' if bit == '0' else '\t' for bit in binary)

def reveal_from_whitespace(ws):
    bits = ''.join('0' if c == ' ' else '1' for c in ws if c in (' ', '\t'))
    chars = [chr(int(bits[i:i+8], 2)) for i in range(0, len(bits), 8)]
    return ''.join(chars)

# Hide
secret = hide_in_whitespace("Secret")
print(repr(secret))  # Shows whitespace

# Reveal
revealed = reveal_from_whitespace(secret)
print(revealed)
```




Ook!

- Ook! is a **joke esoteric** programming language created by **David Morgan-Mar**, designed specifically for **orangutans** (yes, really 🐒)
- It's a humorous take on **Brainfuck**, where each command is replaced with combinations of the word “Ook” followed by **punctuation marks**

Ook! Syntax	Brainfuck Equivalent
Ook. Ook?	>
Ook? Ook.	<
Ook. Ook.	+
Ook! Ook!	-
Ook! Ook.	.
Ook. Ook!	,
Ook! Ook?	[
Ook? Ook!	]



Ook! (Cont)

Ook! ENCODER

★ PLAINTEXT TO CODE IN Ook! (?)

Bank Wongyos



★ SHORT CODE GENERATION (NO Ook) ☐

★ ADD A SPACE SEPARATOR ☒

▶ ENCRYPT

Results



Bank Wongyos

ook.	ook.	ook.	ook.	ook.	ook.	ook.	ook.
ook.	ook.	ook.	ook.	ook.	ook.	ook.	ook.
ook.	ook.	ook.	ook.	ook!	ook?	ook.	ook?
ook.	ook.	ook.	ook?	ook.	ook.	ook.	ook.
ook.	ook.	ook.	ook?	ook.	ook.	ook.	ook.
ook.	ook.	ook.	ook.	ook.	ook.	ook.	ook.
ook.	ook.	ook.	ook?	ook.	ook.	ook.	ook.
ook.	ook.	ook.	ook.	ook.	ook.	ook.	ook.
ook?	ook.	ook?	ook.	ook?	ook.	ook?	ook.
ook!	ook!	ook?	ook!	ook.	ook?	ook.	ook?



Slang-based Encoding and Decoding



Loo-Thai LGBTQ+

- **Loo (ภาษาลู)** is a playful, coded style of writing and speaking used within the **Thai LGBTQ+ community**, especially among **young queer** people online
- It originated as a form of textual or **linguistic obfuscation** for fun, secrecy, or expressing identity
- It's **not an esoteric language** in the strictest programming sense, but rather a sociolect or **slang-based encoding** system

อยากได้ = ลากยุกไล่ดู
ดูดีมาก = ลูดู ลีดู ลากมูก
รำคาญ = ขำรูนลาญคุณ
อย่าเยอะ = หล่าอยู่เลอะยุ
มีแฟนยัง = ลีมู แลนฟุน ลังยุ
โสดจีบได้ = โลดสุดลีบจูบไล่ดู
ไม่ชอบ = ไล่มุยลอบชูป
ไปก่อนนะ = ไลปุหล่อนกุนนะ
เบิร์ดเดย์ = เลิร์ดบู้ด เลดู
ไม่บอก = ไล่มุยหลอกมูก
ก็แล้วแต่ = ล่อกู่ แช่วลู่ แหล่ตุ
ทำอะไรอยู่ = ลำทุมไชรูลู่หุย
ไปเที่ยวกัน = ไลปุ เลี้ยวท่ว ลันกุน
ขี้เกียจ = ลีขู่ เหลียดกูด
ชอบใจ = หลอบชูปไลจุย



Loo-Thai LGBTQ+ (Cont)

หลักการง่าย ๆ ของภาษาลู

1 พยางค์ จะแปลงเป็น 2 พยางค์ทุกคำ

1

คำที่เป็นปกติทั่วไป

คำที่มีสระเสียงยาว (อา อี อื อู เอ
แอ โอ ออ เออ เอีย เอื้อ อัว ๆ ๆ)

STEP 1 : ให้นำ "ล" แทนพยัญชนะเดิม

STEP 2 : จากนั้นนำพยัญชนะจากคำเดิม เติม "สระอุ"

STEP 3 : เสียงวรรณยุกต์ และตัวสะกดเหมือนเดิม

ตัวอย่างเช่น โอเค = โลอ เลอ

WWW.WONGNAI.COM

หลักการง่าย ๆ ของภาษาลู

2

คำที่มีพยัญชนะ ร, ล

STEP 1 : นำ "ซ" มาแทนพยัญชนะเดิม

STEP 2 : นำพยัญชนะเดิมเติม "สระอุ" (สระเสียงยาว)
นำพยัญชนะเดิมเติม "สระอุ" (สระเสียงสั้น)

STEP 3 : เสียงวรรณยุกต์และตัวสะกดจะเหมือนเดิม

ตัวอย่างเช่น ลองรัก = ซองลง ซักรก

WWW.WONGNAI.COM



Loo-Thai LGBTQ+ (Cont)

หลักการง่าย ๆ ของภาษาลู

3

คำที่มีสระอ, สระอุ

- STEP 1 : ให้นำ "หล" แทนพยัญชนะเดิม
- STEP 2 : นำพยัญชนะเดิม เติม "สระอ" ในสระเสียงยาว
เติม "สระอ" ในสระเสียงสั้น
- STEP 3 : คงรูปเสียงวรรณยุกต์ และตัวสะกดไว้เหมือนเดิม

ตัวอย่างเช่น กูด = หลูดกิด

สุก = หลูกสิก

WWW.WONGNAI.COM

หลักการง่าย ๆ ของภาษาลู

4

คำที่มีสระอ, สระอุ

- STEP 1 : ให้นำ "ซ" แทนพยัญชนะ ร, ล ที่มีสระ อ, อุ
- STEP 2 : คำที่มี ร, ล และสระอุ ให้ใช้ "สระอ"
ส่วนคำที่มี ร, ล และสระอ ให้ใช้ "สระอ" จำง่าย ๆ
ตามสระคำสั้น-คำยาว
- STEP 3 : คงรูปเสียงวรรณยุกต์ และตัวสะกดไว้เหมือนเดิม

ตัวอย่างเช่น หนูไม่รู้ = หลู หนี ไหล่มู ซูรี

WWW.WONGNAI.COM



Khuk-Thai Prison

- **Khuk-Thai Cipher** (รหัสคุกไทย) is a **Thai slang** obfuscation technique often used in some **Thai prison**, especially among **Thai youth** and in **meme culture**, to encode text in a humorous or satirical way
- It loosely mimics **prison-style** tattoos or gangster talk, sometimes as a mock-cryptographic method of **hiding real meanings** behind a layer of chaotic **Thai phonetics and symbols**





Khuk-Thai Prison (Cont)

How to

ภาษาตั๋ยมมกุตจบบ้ง่าย
แปลงแบบพยางค์ต่อพยางค์

Step 1: นำ "ซ" มาแทนพยัญชนะเดิม
ของพยางค์

Step 2: นำตัวพยัญชนะตัวที่โดนแทน
มาจับคู่กับ " อ "

เช่น เย็ด - เช็ดหย่อ

- ประโยคคำถาม -
ฝนตกไหม

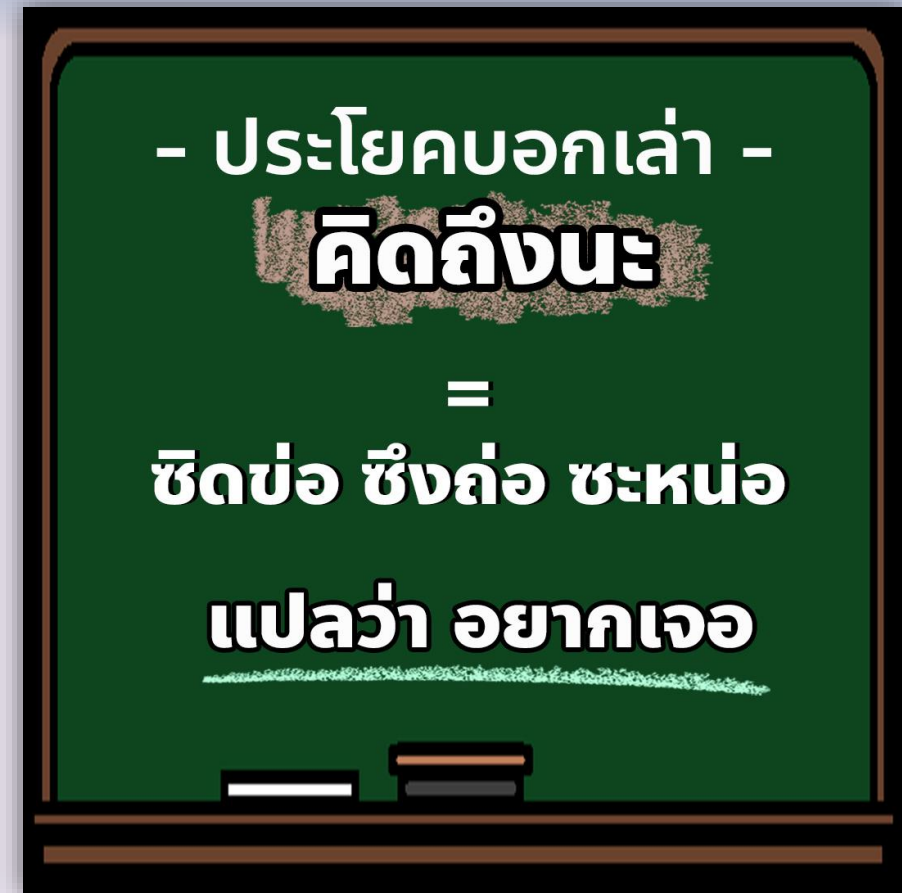
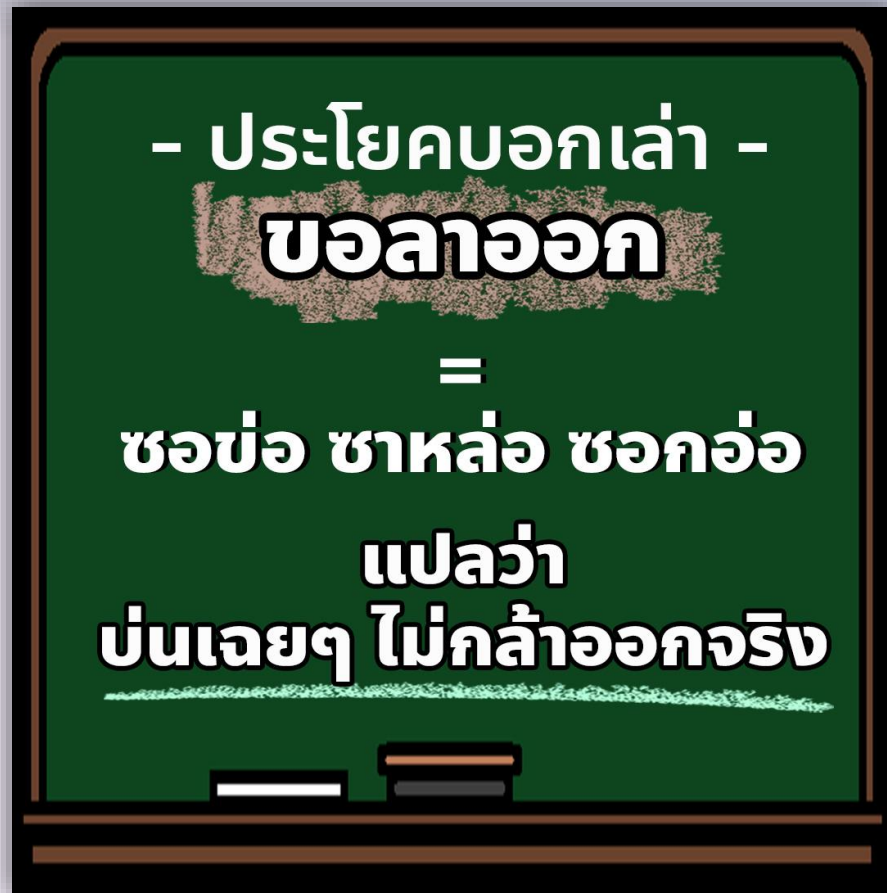
=

ชนฝ่อ ชกต้อ ไชหม่อ

แปลว่า คิดถึงนะ



Khuk-Thai Prison (Cont)





PAPAYA-Thai Slang

- **Papaya Language** (in Thai: ภาษามะละกอ) is a **Thai youth slang style**, especially popular among **Gen Z** and **LGBTQ+** communities on social media platforms like **TikTok**, **Twitter**, and **Threads**
- It's playful, fun, and often used to make serious or emotional expressions **sound cute, funny**, or more **lighthearted**
- Each **word** is broken into **three syllables**:
 - **Syllable 1**: The first consonant of the **original word** + **short vowel "-a" (อะ)**
 - **Syllable 2**: Always the word **"la" (ละ)**, no matter what the original word is
 - **Syllable 3**: The **consonant "k" (ก)** + **the original vowel** sound or something similar to the final part of the original word



PAPAYA-Thai Slang (Cont)

Original	Papaya Form	Meaning
งง (<i>ngong</i>)	งะ ละ กง (<i>nga la kong</i>)	Confused
หิว (<i>hǐw</i>)	ฮะ ละ กิว (<i>ha la giw</i>)	Hungry
รัก (<i>rák</i>)	ระ ละ กัก (<i>ra la gak</i>)	Love
เจ็บ (<i>jèp</i>)	จะ ละ เก็บ (<i>ja la gep</i>)	Hurt
กลัว (<i>glûa</i>)	กะ ละ กัว (<i>ga la gua</i>)	Afraid, scared



PAPAYA-Thai Slang (Cont)



<https://www.youtube.com/watch?v=Svzxtj2YQY>



Pig Latin

- Pig Latin is a playful language game used primarily in English-speaking countries
- It is not a true language, but rather a form of wordplay, often used among children or in informal contexts for fun or privacy
- Basic Rules:
 - If the word begins with a consonant or consonant cluster: Move it to the end of the word and add "ay"
Example: hello → ellohay
 - If the word begins with a vowel (a, e, i, o, u): Just add "way" (or sometimes just "ay") at the end
Example: apple → appleway



Pig Latin (Cont)

■ Python for Pig Latin encoding and decoding

```
def pig_latin(word):  
    vowels = "aeiou"  
    word = word.lower()  
    if word[0] in vowels:  
        return word + "way"  
    else:  
        for i in range(len(word)):  
            if word[i] in vowels:  
                return word[i:] + word[:i] + "ay"  
        return word + "ay" # fallback for no vowels  
  
text = "Hello Pig Latin is fun"  
encoded = " ".join(pig_latin(word) for word in text.split())  
print(encoded)
```

English	Pig Latin
hello	ellohay
banana	ananabay
school	oolschay
eat	eatway
umbrella	umbrellaway
you	ouyay

Symmetric Cryptography





Overviews of Symmetric Cryptography

- **Symmetric cryptography** (also called **secret-key cryptography**) is an algorithm for cryptography that uses **the same cryptographic keys** for both **encryption of plaintext** and **decryption of ciphertext**
- The **keys** represents a **shared secret** between two or more parties that can be used to maintain a private information link

Key-based (one key),
Two-way methods

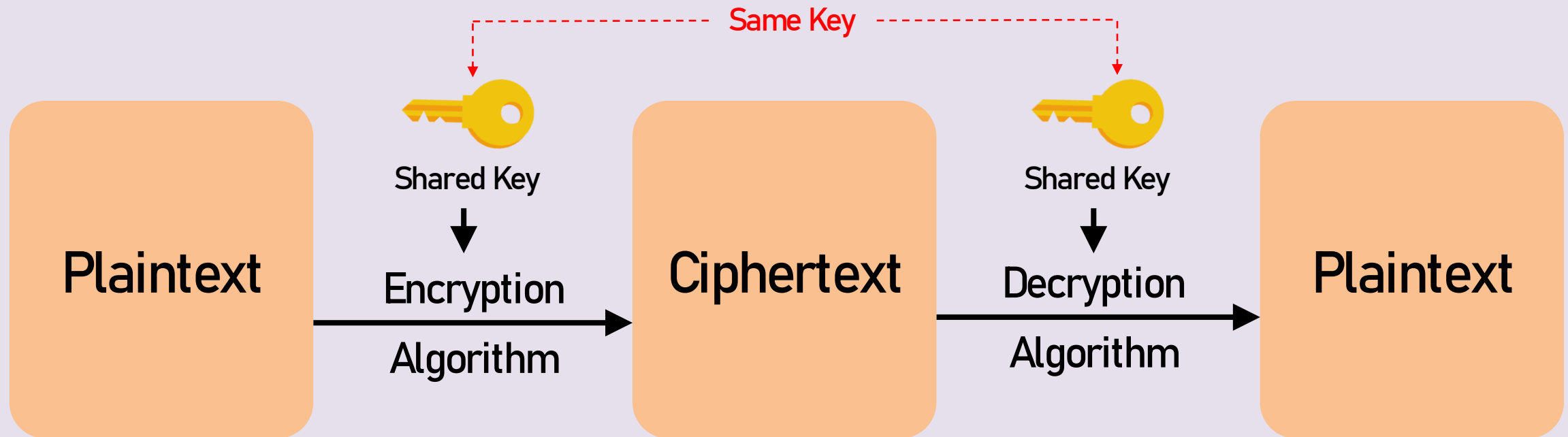


Key Concepts of Symmetric Cryptography

Feature	Description
Single key	Both sender and receiver use the same secret key
Fast and efficient	Especially suitable for encrypting large amounts of data
Key management	The biggest challenge is securely sharing the key between parties
Used in	Secure file storage, VPNs , HTTPS (with TLS), disk encryption, etc.



Process of Symmetric Cryptography





Symmetric Cryptography Algorithms

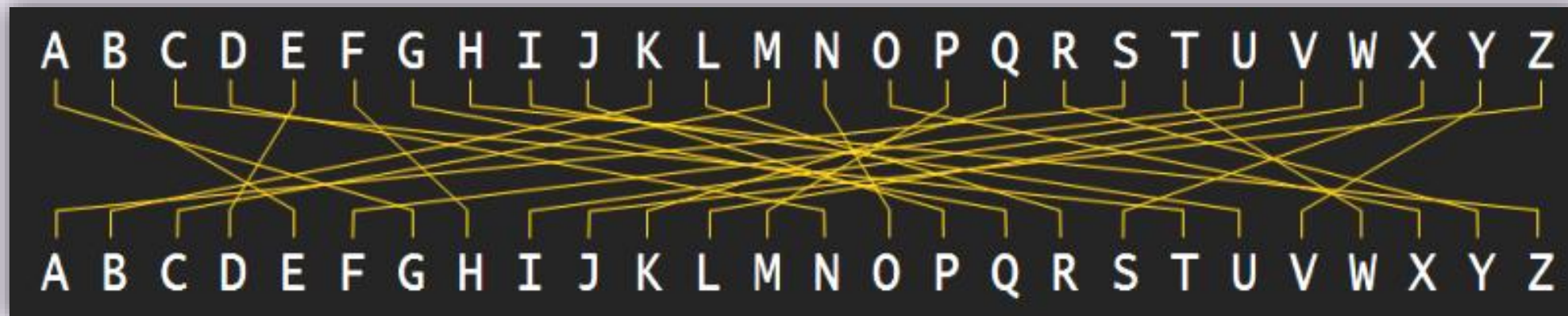
Type	Details	Example of Algorithms
Substitution Cipher	Each letter in the plaintext is replaced by another letter	Vigenère, Affine, Enigma
Block Cipher	Encrypts fixed-size blocks of data (e.g., 64-bit, 128-bit) using the same key	DES, 3DES, AES, Blowfish, Twofish
Stream Cipher	Encrypts data one bit or byte at a time (as a continuous stream) using a keystream generator	XOR, RC4, ChaCha20, Salsa20

Substitution Cipher



Substitution Cipher Overviews

- A **substitution cipher** replaces elements of **plaintext** (typically **letters** or **symbols**) with other characters or values
- These can be used for:
 - **Simple data transformation** (encoding/decoding) – **not secure, no key**
 - **Symmetric cryptography** – uses a **secret key**, intended for confidentiality





Vigenère

- **Vigenère Cipher** is a **polyalphabetic substitution cipher** that uses a **keyword** to determine the shift for each letter in the plaintext
- For each pair:
 - Convert both **letters** to positions: **A=0, B=1, ..., Z=25**
 - Compute: **cipher_char = (plain_char + key_char) % 26**
 - Convert **back to letter**
- Example:
Input: **"HELLO WORLD"**
Key: **"KEY"**
Output: **"RIJVS UYVJN"**



Vigenère (Cont)

Recipe   

Vigenère Encode 

Key
RPCA

Input length: 12
lines: 1     

Bank Wongyos

Output time: 0ms
length: 12
lines: 1     

Sppk Ndpgpdu



Vigenère (Cont)

■ Python for Vigenère encryption and decryption

```
def vigenere_encrypt(plaintext, key):  
    result = []  
    key = key.upper()  
    key_index = 0  
  
    for char in plaintext:  
        if char.isalpha():  
            offset = ord('A') if char.isupper() else ord('a')  
            key_char = ord(key[key_index % len(key)]) - ord('A')  
            encrypted_char = chr((ord(char.upper()) - ord('A') + key_char) % 26 + offset)  
            result.append(encrypted_char if char.isupper() else encrypted_char.lower())  
            key_index += 1  
        else:  
            result.append(char)  
    return ''.join(result)
```




Vigenère (Cont)

```
def vigenere_decrypt(ciphertext, key):  
    result = []  
    key = key.upper()  
    key_index = 0  
  
    for char in ciphertext:  
        if char.isalpha():  
            offset = ord('A') if char.isupper() else ord('a')  
            key_char = ord(key[key_index % len(key)]) - ord('A')  
            decrypted_char = chr((ord(char.upper()) - ord('A') - key_char + 26) % 26 + offset)  
            result.append(decrypted_char if char.isupper() else decrypted_char.lower())  
            key_index += 1  
        else:  
            result.append(char)  
    return ''.join(result)
```



Affine

- Affine cipher is a classical **monoalphabetic substitution** cipher that uses a **mathematical function** to encrypt each letter
- It's an example of a symmetric cryptographic technique, but it is **not secure** by modern standards
- The **encryption** function is:
 $E(x) = (a * x + b) \bmod m$
- The **decryption** function is:
 $D(x) = a^{-1}(x - b) \bmod m$
- For decryption to work
a must have a modular inverse mod **m**, i.e., $\gcd(a, m) = 1$

Where:

x is the letter's position in the alphabet (e.g., A=0, B=1, ..., Z=25)

a and **b** are keys of the cipher (with **a** and **m** coprime)

m is the size of the alphabet (**typically 26 for English**)

a is the multiplicative key

b is the additive key

a^{-1} is the modular inverse of **a** modulo **m**



Affine (Cont)

■ Example

- Let: $a = 5$, $b = 8$, $m = 26$
- Plaintext: HELLO
- Step-by-step for $H = 7$:
- $E(H) = (5 * 7 + 8) \bmod 26 = 43 \bmod 26 = 17 = R$
- Result: RCLLA

Results       

$f(x) = 5x + 8$

 = ABCDEFGHIJKLMNOPQRSTUVWXYZ

RCLLA



Affine (Cont)

AFFINE ENCODER

★ AFFINE PLAIN TEXT ?

Bank wongyos

★ ALPHABET ABCDEFGHIJKLMNOPQRSTUVWXYZ

Ⓢ VALUES FOR COEFFICIENTS OF AFFINE FUNCTION $AX+B$

★ A COEFFICIENT

5

★ B COEFFICIENT

8

► ENCRYPT

Results



$$f(x)=5x+8$$

A_B = ABCDEFGHIJKLMNOPQRSTUVWXYZ

Nivg Oavmyau

Affine Cipher - dCode

Tag(s) : Substitution Cipher



Affine (Cont)

■ Python for Affine encryption and decryption

```
def mod_inverse(a, m):  
    for x in range(1, m):  
        if (a * x) % m == 1:  
            return x  
    return None  
  
def affine_encrypt(text, a, b):  
    result = ''  
    for char in text.upper():  
        if char.isalpha():  
            x = ord(char) - ord('A')  
            result += chr(((a * x + b) % 26) + ord('A'))  
        else:  
            result += char  
    return result
```



Affine (Cont)

```
def affine_decrypt(cipher, a, b):
    result = ''
    a_inv = mod_inverse(a, 26)
    if a_inv is None:
        return "Invalid 'a' value; no modular inverse exists."
    for char in cipher.upper():
        if char.isalpha():
            y = ord(char) - ord('A')
            result += chr(((a_inv * (y - b)) % 26) + ord('A'))
        else:
            result += char
    return result

# Example usage
plaintext = "HELLO"
a, b = 5, 8
cipher = affine_encrypt(plaintext, a, b)
print("Encrypted:", cipher)
print("Decrypted:", affine_decrypt(cipher, a, b))
```



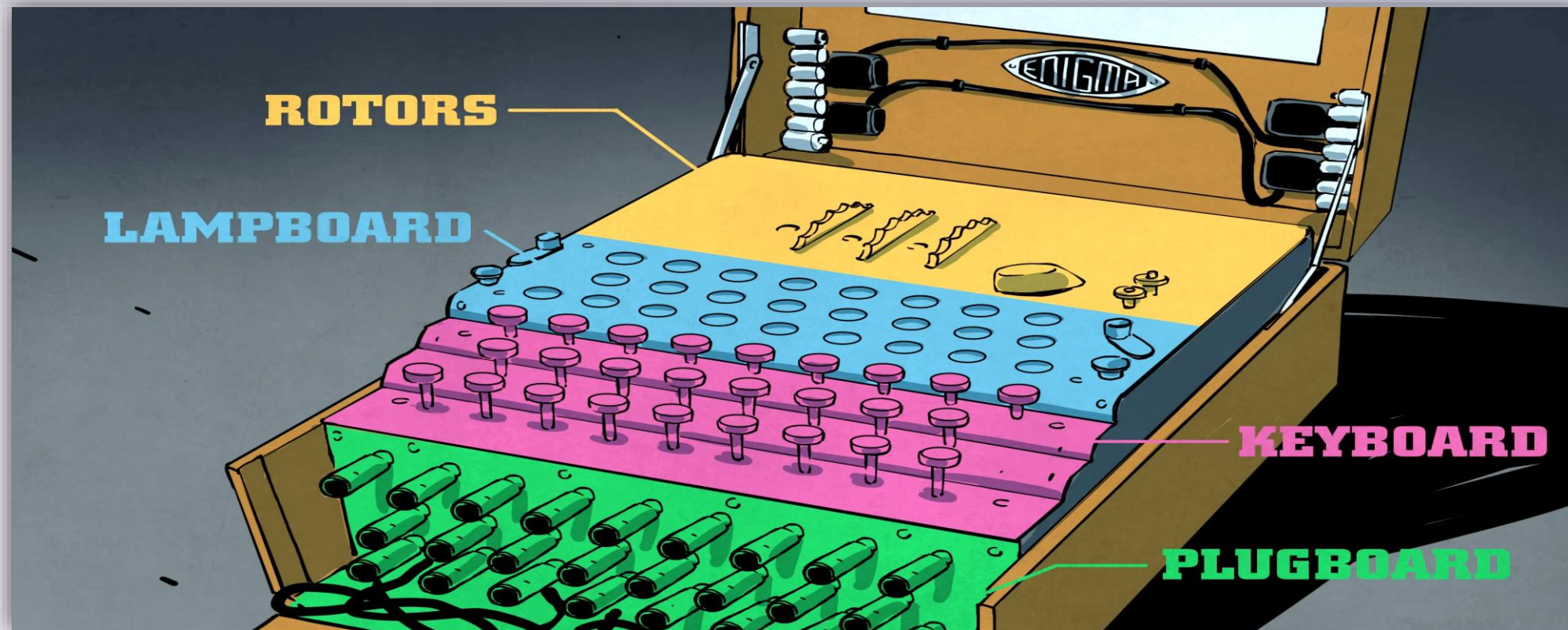
Enigma

- **Enigma machine** was a complex **electro-mechanical rotor** cipher machine used primarily by **Nazi Germany** during **World War II** for secure military communications
- It is one of the **most famous symmetric encryption** (using **substitution** with **polyalphabetic** behavior) in history and was considered unbreakable—until Allied cryptanalysts famously cracked it
- **Alan Turing** and his team at **Bletchley Park** developed the **Bombe**, an electromechanical device to discover Enigma settings





Enigma (Cont)



<https://hackaday.com/2017/08/22/the-enigma-enigma-how-the-enigma-machine-worked/>



Enigma (Cont)

- **Plugboard (Steckerbrett)**
 - Swaps letters in pairs (e.g., A ↔ G)
- **Rotors (Walzen)**
 - Rotate after each key press
 - Each rotor performs a monoalphabetic substitution
 - Multiple rotors = polyalphabetic substitution
- **Reflector (Umkehrwalze)**
 - Sends the current back through the rotors (creates symmetry)
 - No letter ever encrypts to itself
- **Daily Settings**
 - Rotor order
 - Ring settings
 - Plugboard connections
 - Initial rotor positions

How Enigma Works





Enigma (Cont)

ENIGMA ENCODER AND DECODER

★ MESSAGE TO TYPE ON THE ENIGMA MACHINE

Bank Wongyos

★ PRESERVE DIGITS, PUNCTUATION, ETC. ☐

★ ROTORS ⚙ (FROM LEFT TO RIGHT) (WALZENLAGE) I - II - III

★ REFLECTOR ⇄ (UMKEHRWALZE) B

★ POSITIONS OF RINGS ◦ (RINGSTELLUNG) A, A, A

★ POSITIONS OF ROTORS ⚡ (GRUNDSTELLUNG) 1, 1, 1

★ CONNECTIONS PAIRS ⚡ (STECKERVERBINDUNGEN)

Results

	⚡	⚙	⚙	⚙
	B	I	II	III
	◦01=A	◦01=A	◦01=A	
	⚡01=A	⚡01=A	⚡01=A	

ADOHQKRLGSJ

Recipe

Enigma

Model: 3-rotor

Left-hand rotor: EKMFLGDQVZNTOWYHXU...

Left-hand rotor ring setting: A

Left-hand rotor initial value: A

Middle rotor: AJDKSIRUXBLHWTMCQG...

Middle rotor ring setting: A

Middle rotor initial value: A

Right-hand rotor: BDFHJLCPRTXVZNYEIW...

Right-hand rotor ring setting: A

Right-hand rotor initial value: A

Reflector: AY BR CU DH EQ FS GL IP JX KN MO TZ...

Plugboard

Strict output

Input

Bank Wongyos

Output

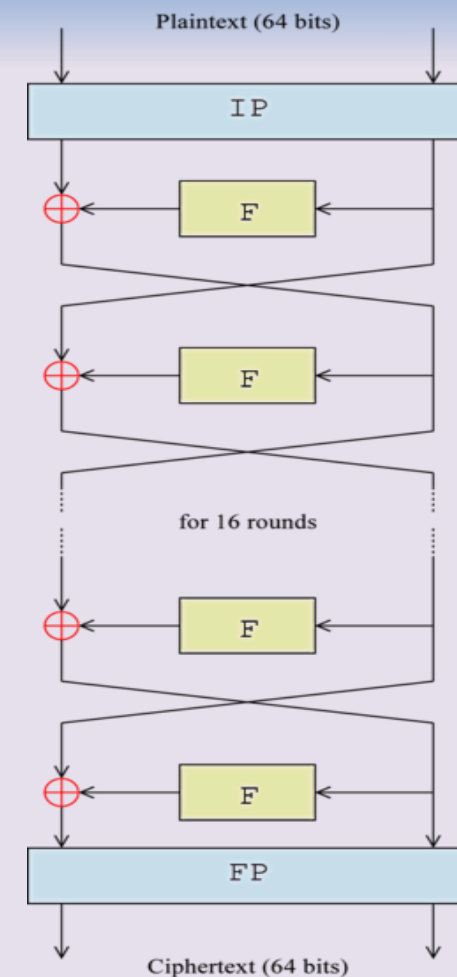
ADOHQ KRLGS J

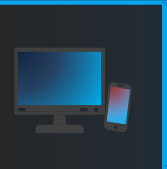
Block Cipher



DES

- **DES (Data Encryption Standard)** is a **symmetric-key block cipher** developed in the early **1970s** by **IBM** and adopted as a federal standard by the **U.S.** National Institute of Standards and Technology (**NIST**) in **1977**
- It was one of the most **influential early encryption** algorithms and paved the way for modern cryptography
- **DES** uses a **56-bit key**, typically input as **64 bits (8 bytes)** where every 8th bit is for parity





DES (Cont)

■ Overview of DES

Feature	Description
Type	Symmetric-key block cipher
Block Size	64 bits (8 bytes)
Key Size	56 bits (effective)
Rounds	16 rounds of Feistel network
Developed By	IBM (as “Lucifer”), refined by NSA
Adopted By	NIST (as a federal standard in 1977)
Status	Deprecated (replaced by AES)



DES (Cont)

- **Initialization Vector (IV):**
 - Used only in certain modes like **CBC, CFB, OFB**
 - Must be the **same size** as the **block size: 8 bytes (64 bits)**
 - Must be: (1) **randomly** generated for encryption and (2) **shared** or known for decryption

- **DES Modes of Operation:**

Mode	Description	Requires IV
ECB	Electronic Codebook – simplest, but not secure; identical blocks = same ciphertext	No
CBC	Cipher Block Chaining – XORs plaintext block with previous ciphertext block	Yes
CFB	Cipher Feedback – converts block cipher into a stream cipher	Yes
OFB	Output Feedback – like CFB, but IV is updated independently	Yes



DES (Cont)

- DES Weaknesses

Vulnerability	Description
Short key size	56-bit key can be brute-forced with modern computing
Known attacks	Linear cryptanalysis, differential cryptanalysis
Deprecated	Officially withdrawn as a standard by NIST in 2005



DES (Cont)

Recipe		Input
DES Encrypt		Bank Wongyos
Key deadbeef12345678	HEX ▾	
IV 12345678deadbeef	HEX ▾	
Mode CBC	Input Raw	
Output Hex		
		abc 12 1
		Output
		2593e35d7bcc373562ebb3ef3615704a



DES (Cont)

■ Python for DES encryption and decryption

```
from Crypto.Cipher import DES
from Crypto.Util.Padding import pad, unpad
import binascii

key = b'8bytekey' # 8 bytes = 64 bits
cipher = DES.new(key, DES.MODE_ECB)

plaintext = b"BankWongyos123"
padded = pad(plaintext, DES.block_size)
ciphertext = cipher.encrypt(padded)
print("Encrypted:", binascii.hexlify(ciphertext))

decipher = DES.new(key, DES.MODE_ECB)
decrypted = unpad(decipher.decrypt(ciphertext), DES.block_size)
print("Decrypted:", decrypted.decode())
```



DES (Cont)

■ Python for DES encryption and decryption with IV

```
from Crypto.Cipher import DES
from Crypto.Random import get_random_bytes
from Crypto.Util.Padding import pad, unpad
import binascii

# Key must be 8 bytes
key = b'8bytekey'

# --- ECB Mode (No IV) ---
cipher_ecb = DES.new(key, DES.MODE_ECB)
plaintext = b'Bank Wongyos!'
padded_text = pad(plaintext, DES.block_size)
ciphertext_ecb = cipher_ecb.encrypt(padded_text)
print("ECB Encrypted:", binascii.hexlify(ciphertext_ecb))
```



DES (Cont)

```
# Decrypt
decipher_ecb = DES.new(key, DES.MODE_ECB)
decrypted_ecb = unpad(decipher_ecb.decrypt(ciphertext_ecb), DES.block_size)
print("ECB Decrypted:", decrypted_ecb.decode())

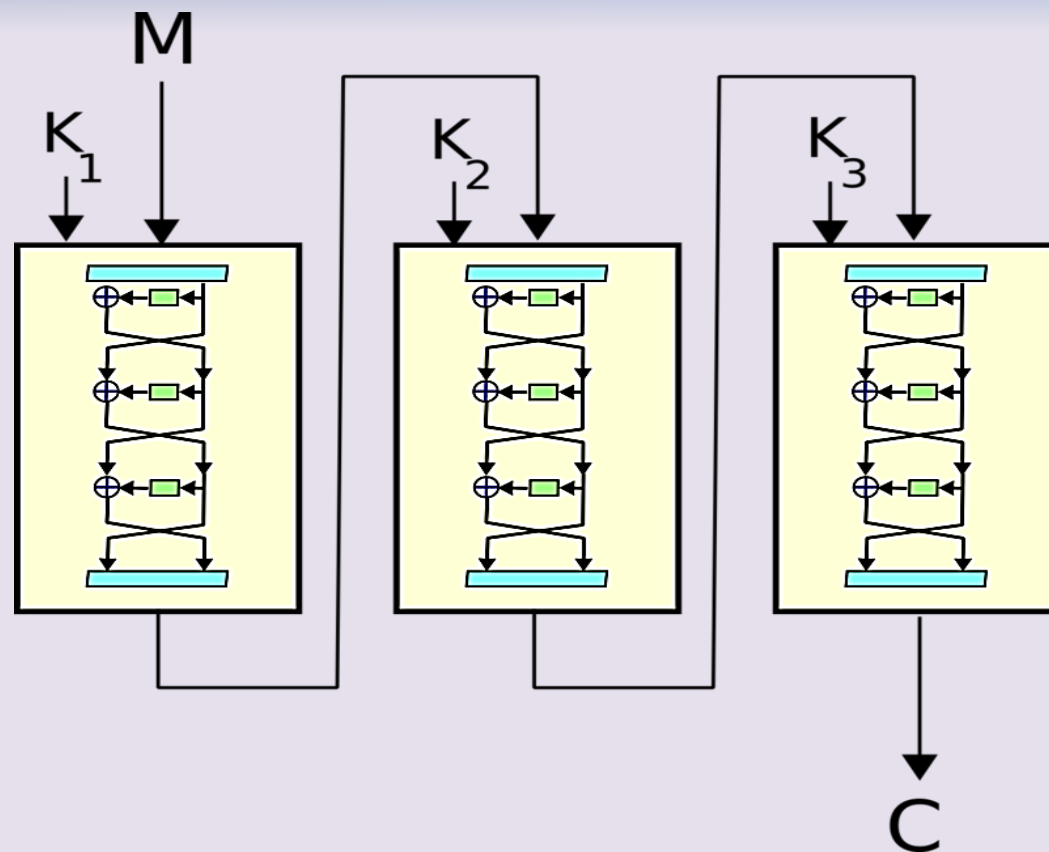
# --- CBC Mode (With IV) ---
iv = get_random_bytes(8) # 8 bytes for DES
cipher_cbc = DES.new(key, DES.MODE_CBC, iv)
ciphertext_cbc = cipher_cbc.encrypt(pad(plaintext, DES.block_size))
print("CBC Encrypted:", binascii.hexlify(ciphertext_cbc))
print("IV used:", binascii.hexlify(iv))

# Decrypt
decipher_cbc = DES.new(key, DES.MODE_CBC, iv)
decrypted_cbc = unpad(decipher_cbc.decrypt(ciphertext_cbc), DES.block_size)
print("CBC Decrypted:", decrypted_cbc.decode())
```



3DES

- **3DES (Triple DES)** is an encryption algorithm designed to **improve upon DES** by applying the DES cipher three times to each data block
- It's **more secure than DES** but slower and considered deprecated for most modern use





3DES (Cont)

■ Overview of 3DES (Triple DES)

Feature	Description
Full Name	Triple Data Encryption Standard
Block Size	64 bits (8 bytes)
Key Sizes	112 bits (2 keys) or 168 bits (3 keys)
Number of Passes	3 (Encrypt-Decrypt-Encrypt or EDE)
Modes	ECB, CBC, CFB, OFB (like DES)
Library Support	Yes (e.g., pycryptodome)
Security Status	Deprecated (use AES instead)



3DES (Cont)

■ Python for 3DES encryption and decryption

```
from Crypto.Cipher import DES3
from Crypto.Random import get_random_bytes
from Crypto.Util.Padding import pad, unpad
import binascii

# Ensure key is 16 (2-key) or 24 (3-key) bytes long
key = DES3.adjust_key_parity(get_random_bytes(24)) # 24 bytes = 3-key 3DES
iv = get_random_bytes(8) # Block size for DES/3DES is 8 bytes
plaintext = b'Bank Wongyos Cyber Team'

# --- Encryption ---
cipher = DES3.new(key, DES3.MODE_CBC, iv)
ciphertext = cipher.encrypt(pad(plaintext, DES3.block_size))
print("Encrypted (Hex):", binascii.hexlify(ciphertext))
print("IV (Hex):", binascii.hexlify(iv))
```



3DES (Cont)

```
# --- Decryption ---  
decipher = DES3.new(key, DES3.MODE_CBC, iv)  
decrypted = unpad(decipher.decrypt(ciphertext), DES3.block_size)  
print("Decrypted:", decrypted.decode())
```



AES

- **Advanced Encryption Standard (AES)** is a **symmetric block cipher** algorithm widely used for encrypting sensitive data
- AES encrypts data in blocks of **128 bits** using **keys** of **128**, **192**, or **256 bits**
- It's a **secure** and **efficient** algorithm, considered a global standard for data security
- **Overview** of AES

Feature	Description
Algorithm Name	AES (Advanced Encryption Standard)
Block Size	128 bits (16 bytes)





AES (Cont)

Feature	Description
Key Sizes	128, 192, or 256 bits (16, 24, or 32 bytes)
Rounds	10 (AES-128), 12 (AES-192), 14 (AES-256)
Modes	ECB, CBC, CFB, OFB, CTR, GCM
Security	Strong (especially AES-256), widely used
Library	Available via pycryptodome, cryptography, etc.





AES (Cont)

- **AES Modes** of Operation (Common)

Mode	Description	IV Required?
ECB	Simplest, but insecure; patterns leak	No
CBC	Cipher Block Chaining; better for security	Yes
CFB	Cipher Feedback; converts to a stream cipher	Yes
OFB	Output Feedback; like CFB, but less dependent on plaintext	Yes
CTR	Counter mode; uses a counter instead of chaining	Yes
GCM	Galois/Counter Mode; provides confidentiality + authenticity (AEAD)	Yes



AES (Cont)

Recipe

^

📁

🗑️

AES Encrypt

^

🛑

⏸

Key

deadbeef87654321deadbe...

HEX ▾

IV

1234567887654321123456...

HEX ▾

Mode

CBC

Input

Raw

Output

Hex

Input

Bank Wongyos|

ABC 12

≡

1

Output

be0fb25958805e75e06fc1e7177b5df5|

Chapter 6

Cryptography

Copyright © 2026 By Wongyos Keardsri



AES (Cont)

■ Python for AES encryption and decryption

```
from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes
from Crypto.Util.Padding import pad, unpad
import binascii

# Key must be 16 (AES-128), 24 (AES-192), or 32 (AES-256) bytes
key = get_random_bytes(32) # AES-256
iv = get_random_bytes(16) # AES block size is 16 bytes
plaintext = b'Bank Wongyos Cyber Academy'

# --- Encryption ---
cipher = AES.new(key, AES.MODE_CBC, iv)
ciphertext = cipher.encrypt(pad(plaintext, AES.block_size))
print("Encrypted (Hex):", binascii.hexlify(ciphertext))
print("IV (Hex):", binascii.hexlify(iv))
```



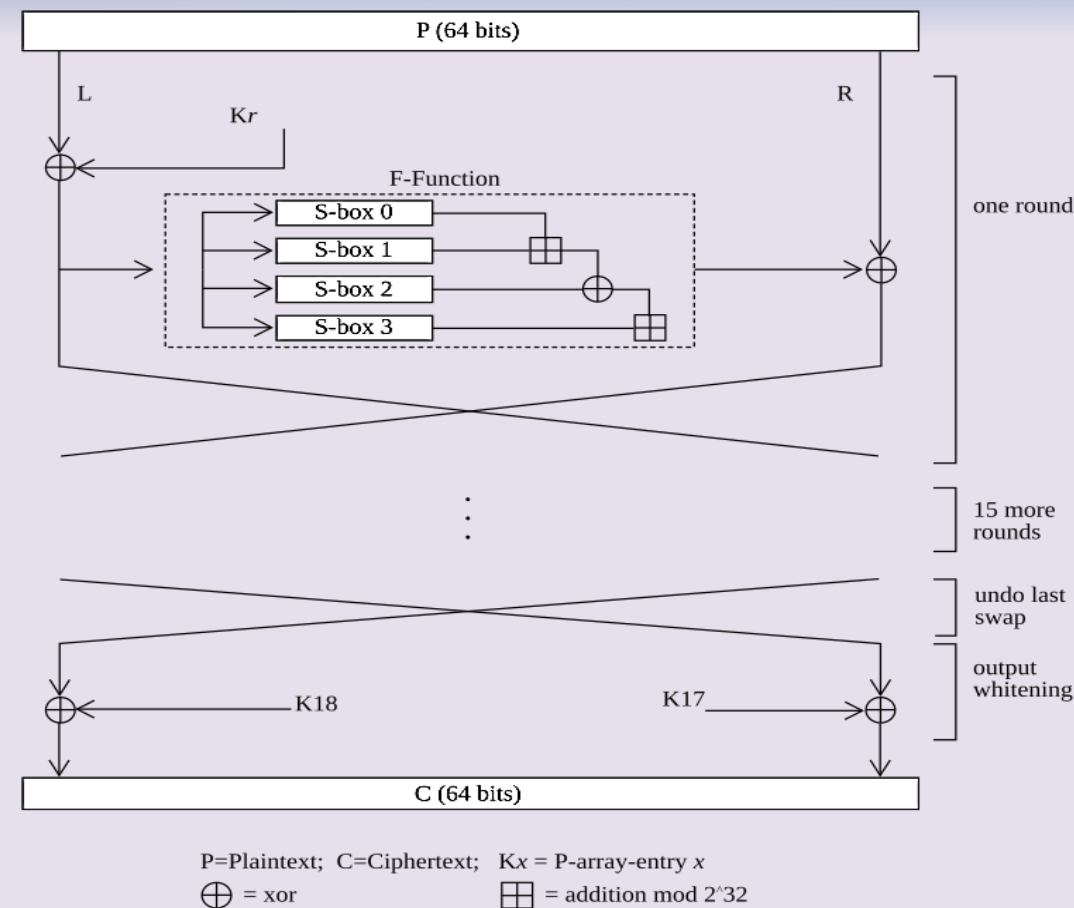
AES (Cont)

```
# --- Decryption ---  
decipher = AES.new(key, AES.MODE_CBC, iv)  
decrypted = unpad(decipher.decrypt(ciphertext), AES.block_size)  
print("Decrypted:", decrypted.decode())
```



Blowfish

- **Blowfish** is a **symmetric-key block cipher**, designed by **Bruce Schneier** in 1993
- It's known for its **speed** and **simplicity**, making it a good choice for software applications that require **fast encryption**
- Its use has declined due to the emergence of more secure and efficient algorithms like **AES**





Blowfish (Cont)

■ Overview of Blowfish

Feature	Description
Algorithm Name	Blowfish
Type	Symmetric-key block cipher
Block Size	64 bits (8 bytes)
Key Size	Variable (32 to 448 bits)
Rounds	16
Designer	Bruce Schneier (1993)
Performance	Very fast in software
Status	Secure but outdated for some modern applications



Blowfish (Cont)

- **Blowfish Modes** of Operation (like **AES**)

Mode	Description	IV Required?
ECB	Insecure due to pattern leakage	No
CBC	Adds chaining with IV	Yes
CFB	Turns Blowfish into a stream cipher	Yes
OFB	Output feedback mode	Yes



Blowfish (Cont)

Recipe

Blowfish Encrypt

Key
1234567812345678
HEX ▾

IV
1234567812345678
HEX ▾

Mode
CBC

Input
Raw

Output
Hex

Input

Bank Wongyos

ABC 12 1

Output

e18280a6a76e803cccc25fe7e30e3225



Blowfish (Cont)

■ Python for Blowfish encryption and decryption

```
from Crypto.Cipher import Blowfish
from Crypto.Random import get_random_bytes
from Crypto.Util.Padding import pad, unpad
import binascii

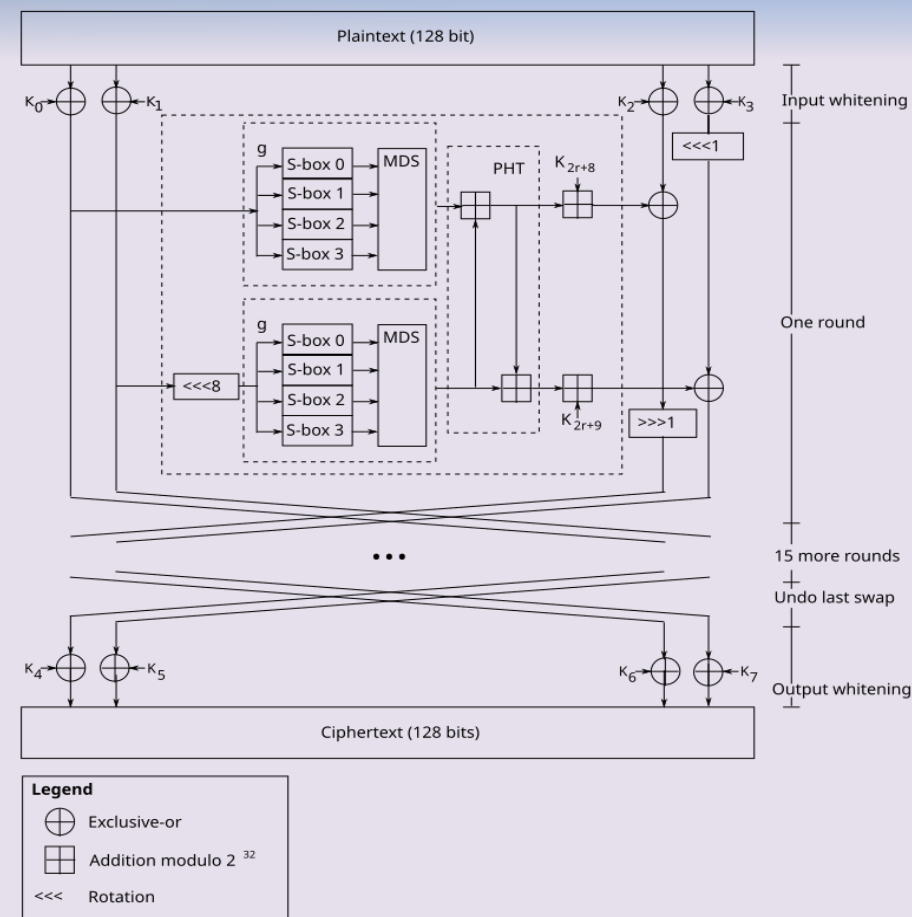
key = get_random_bytes(16) # Example 128-bit key
iv = get_random_bytes(8) # Blowfish block size is 8 bytes
data = b'Confidential Info - Wongyos'
cipher = Blowfish.new(key, Blowfish.MODE_CBC, iv)
ciphertext = cipher.encrypt(pad(data, Blowfish.block_size))
print("Encrypted (Hex):", binascii.hexlify(ciphertext))

# Decryption
decipher = Blowfish.new(key, Blowfish.MODE_CBC, iv)
decrypted = unpad(decipher.decrypt(ciphertext), Blowfish.block_size)
print("Decrypted:", decrypted.decode())
```



Twofish

- **Twofish** is a **symmetric key block cipher** with a block size of **128 bits** and **key** sizes up to **256 bits**
- It was one of the **five finalists** of the **Advanced Encryption Standard contest**, but it was not selected for standardization
- **Twofish** is related to the earlier block cipher Blowfish





Twofish (Cont)

■ Overview of Twofish

Feature	Description
Algorithm Name	Twofish
Type	Symmetric-key block cipher
Block Size	128 bits (16 bytes)
Key Size	Variable (128, 192, or 256 bits)
Rounds	16
Designer	Bruce Schneier and the Counterpane Team (1998)
Performance	Fast and flexible; good for both hardware and software
Status	Secure and free to use; alternative to AES



Twofish (Cont)

■ Twofish Modes of Operation

Mode	Description	IV Required?
ECB	Simple, insecure for patterned data	No
CBC	Chained blocks, better security	Yes
CFB	Stream-like cipher mode	Yes
OFB	Feedback stream cipher	Yes
CTR	Counter-based stream cipher	Yes



Twofish (Cont)

■ Python for Twofish encryption and decryption

```
from Crypto.Cipher import Twofish
from Crypto.Random import get_random_bytes
from Crypto.Util.Padding import pad, unpad
import binascii

key = get_random_bytes(16) # 128-bit key
iv = get_random_bytes(16) # Twofish block size is 16 bytes
data = b'Hello Wongyos Keardsri'
cipher = Twofish.new(key, Twofish.MODE_CBC, iv)
ciphertext = cipher.encrypt(pad(data, Twofish.block_size))
print("Encrypted (Hex):", binascii.hexlify(ciphertext))

# Decryption
decipher = Twofish.new(key, Twofish.MODE_CBC, iv)
decrypted = unpad(decipher.decrypt(ciphertext), Twofish.block_size)
print("Decrypted:", decrypted.decode())
```

Stream Cipher



XOR

- **XOR (Exclusive OR)** is a fundamental **bitwise operation** used extensively in symmetric cryptography, especially for:
 - Encryption/decryption in **stream ciphers**
 - Simple **obfuscation**
- Encryption: **ciphertext = plaintext $\oplus$ key**
- Decryption: **plaintext = ciphertext $\oplus$ key**
- Example:
Plaintext: **10101010**
Key: **11001100**
Ciphertext: **01100110**



XOR (Cont)

- **XOR** compares two bits:
 - If bits are the **same** → result is **0**
 - If bits are **different** → result is **1**

A	B	$A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0



XOR (Cont)

Recipe

XOR

Key

13

DECIMAL ▾

Scheme

Standard

☐ Null preserving

Input

Bank Wongyos

RBC 12 1

Tt Raw Bytes LF

Output

01cf-Zbcjtb~



XOR (Cont)

- Python for XOR encryption and decryption

```
def xor_encrypt_decrypt(data: str, key: str) -> str:  
    return ''.join(chr(ord(c) ^ ord(k)) for c, k in zip(data, key *  
        (len(data) // len(key) + 1)))
```

```
# Example usage  
plaintext = "HELLO"  
key = "KEY"  
ciphertext = xor_encrypt_decrypt(plaintext, key)  
print("Encrypted:", ciphertext)  
decrypted = xor_encrypt_decrypt(ciphertext, key)  
print("Decrypted:", decrypted)
```



RC4

- RC4 (Rivest Cipher 4, also known as ARC4 or ARCFOUR) is a stream cipher
- It is remarkable for its simplicity and speed in software, multiple vulnerabilities have been discovered in RC4, rendering it insecure
- It is especially vulnerable when the beginning of the output keystream is not discarded, or when nonrandom or related keys are used
- Particularly problematic uses of RC4 have led to very insecure protocols such as WEP



RC4 (Cont)

- RC4 Weaknesses

Issue	Description
Key leakage	Poorly initialized keys leak information
Biases in output	Early keystream bytes are biased
Broken in practice	Used in WEP, easily cracked
Deprecated	Banned by IETF and major security bodies



RC4 (Cont)

Recipe

RC4

Passphrase
RPCACyberClub

Input format
Latin1

Output format
Hex

Input

Bank Wongyos

Output

00751eb88965cb12f3ae865d



RC4 (Cont)

■ Python for RC4 encryption and decryption

```
from Crypto.Cipher import ARC4
from Crypto.Hash import SHA

key = b'secret_key'
message = b'Hello Wongyos'
key = SHA.new(key).digest()
cipher = ARC4.new(key)
ciphertext = cipher.encrypt(message)
print("Encrypted:", ciphertext)

decipher = ARC4.new(key)
plaintext = decipher.decrypt(ciphertext)
print("Decrypted:", plaintext.decode())
```



ChaCha20

- ChaCha20 is a **performant** and **lightweight**, yet highly secure, **256-bit stream cipher** (new window) used to encrypt and decrypt data
- It's often used to **secure data transmitted online**, such as **emails**, **messages**, **web traffic**, and files being uploaded to the cloud
- ChaCha20 is one of the most secure and **modern stream ciphers** available today, trusted by major platforms for its **simplicity**, **speed**, and **robustness**
- ChaCha20 operates on **512-bit internal state matrix**, updated with a series of quarter-round functions





ChaCha20 (Cont)

Recipe

ChaCha

Key
1234567812345678123456...
HEX ▾

Nonce
1234567812345678
HEX ▾

Counter
10

Rounds
20

Input
Raw

Output
Hex

Input

Bank Wongyos

ABC 12 1

Output

87 05 33 c6 e1 11 e8 03 e4 e1 24 94



ChaCha20 (Cont)

■ Python for ChaCha20 encryption and decryption

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.backends import default_backend
import os

# Key and nonce (must be 32 bytes and 12 bytes respectively)
key = os.urandom(32)
nonce = os.urandom(12)
plaintext = b'Hello Wongyos, this is ChaCha20 encryption!'

# Create cipher
algorithm = algorithms.ChaCha20(key, nonce)
cipher = Cipher(algorithm, mode=None, backend=default_backend())
encryptor = cipher.encryptor()
ciphertext = encryptor.update(plaintext)
```



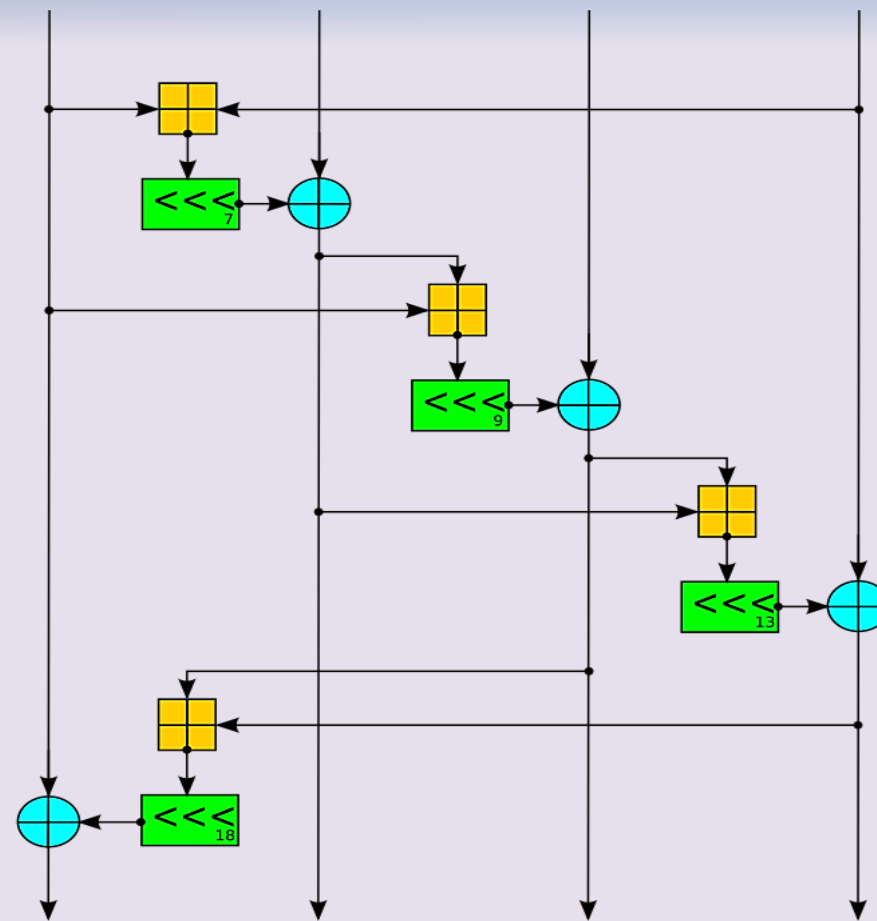
ChaCha20 (Cont)

```
# Decryption
decryptor = cipher.decryptor()
decrypted = decryptor.update(ciphertext)
print("Ciphertext:", ciphertext)
print("Decrypted:", decrypted.decode())
```



Salsa20

- **Salsa20** and the closely related **ChaCha** are stream ciphers developed by **Daniel J. Bernstein**
- **Salsa20**, the original cipher, was designed in **2005**, then later submitted to the **eSTREAM European Union** cryptographic validation process by **Bernstein**
- **ChaCha** is a modification of **Salsa20** published in **2008**





Salsa20 (Cont)

Recipe

^



Salsa20

^



Key

1234567812345678123456...

HEX ▾

Nonce

1234567812345678

HEX ▾

Counter

5

Rounds

20

Input

Raw

Output

Hex

Input

Bank Wongyos

abc 12 ≡ 1

Output

c0 a8 0f 3c 82 cc ce 92 dc df f6 74

Chapter 6

Cryptography

Copyright © 2026 By Wongyos Keardsri



Salsa20 (Cont)

■ Python for Salsa20 encryption and decryption

```
from nacl.secret import SecretBox
from nacl.utils import random

key = random(SecretBox.KEY_SIZE)
box = SecretBox(key)
message = b"Hello Wongyos - Salsa20 stream cipher!"
nonce = random(SecretBox.NONCE_SIZE)

encrypted = box.encrypt(message, nonce)
decrypted = box.decrypt(encrypted)

print("Encrypted:", encrypted)
print("Decrypted:", decrypted.decode())
```

Asymmetric Cryptography





Overviews of Asymmetric Cryptography

- **Asymmetric cryptography** or **public-key cryptography**, or, is a cryptographic system that uses pairs of keys:
 - **Public Keys**, which may be **disseminated widely**
 - **Private Keys**, which are **known only to the owner**
- The generation of keys depends on cryptographic algorithms based on **mathematical functions**

Key-based (two keys),
Two-way methods

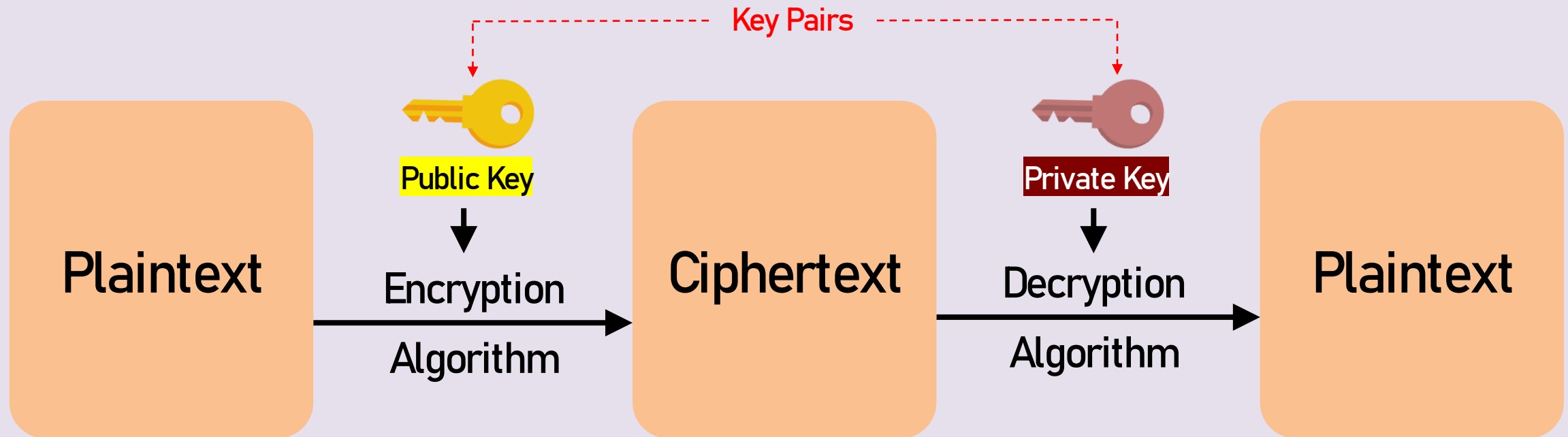


Key Concepts of Asymmetric Cryptography

Feature	Description
Two Keys	Public and private keys are mathematically related but not identical
Two-way Encryption	Data encrypted with one key can only be decrypted with the other
Secure Sharing	The public key can be shared freely; the private key remains secret
Digital Signatures	A message signed with the private key can be verified with the public key



Process of Asymmetric Cryptography





Asymmetric Cryptography Algorithms

Algorithm	Description	Key Size	Notes
RSA	Rivest–Shamir–Adleman	1024–4096 bits	Most widely used
ElGamal	Based on discrete logarithms	Variable	Used in PGP
ECC	Elliptic Curve Cryptography	256–512 bits	Shorter keys, faster performance
DSA	Digital Signature Algorithm	1024+ bits	Used for digital signatures
Diffie-Hellman	Key exchange protocol	Variable	Used to agree on a shared key



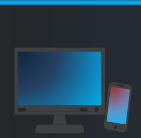
RSA

- RSA is the designer family name of **Ron Rivest**, **Adi Shamir**, and **Leonard Adleman**
- RSA is a well-known **asymmetric cryptographic algorithm** used by modern computers to encrypt and decrypt messages
- RSA have **two different keys**:
 - **Public Key**: the keys can be given to anyone
 - **Private Key**: the key must be kept private
- The algorithm is based on the finding of the **factors** of a large composite number called **prime factorization**



RSA Key Generation

- Choose two different large random prime numbers p and q
- Calculate the modulus: $n = pq$
- Calculate the totient: $\phi(n) = (p - 1)(q - 1)$
- Choose the public key exponent e such that e is an integer prime in range $1 < e < \phi(n)$
- Calculate and choose the private key exponent d such that d is an integer by considering $de \bmod \phi(n) \equiv 1$ or $de = 1 + x(\phi(n))$ for some integer x
- The public key is (n and e)
- The private key is (n and d)



RSA Encryption and Decryption

- Encryption:

$$C = M^e \bmod n \text{ (using public key = } e, n)$$

C = Ciphertext

M = Message in integer form

- Decryption:

$$M = C^d \bmod n \text{ (using private key = } d, n)$$

M = Original plaintext



RSA Processing

- Choose two random prime numbers p and q
 $p = 61$ and $q = 53$
- Compute $n = pq$
 $n = 61 * 53 = 3233$
- Compute $\phi(n) = (p - 1)(q - 1)$
 $\phi(n) = (61 - 1)(53 - 1) = 3120$
- Choose e coprime to 3120
 $e = 17$
- Compute and choose $d \rightarrow de = 1 + x(\phi(n))$
 $d = 2753 \rightarrow 17 * 2753 = 1 + 15(3120) = 46801$



RSA Processing (Cont)

- The public key is ($n = 3233$, $e = 17$)
- The private key is ($n = 3233$, $d = 2753$)
- To encrypt $m = 123$, we can calculate $c = m^e \bmod n$
 $c = 123^{17} \bmod 3233 = 855$
- To decrypt $c = 855$, we can calculate $m = c^d \bmod n$
 $m = 855^{2753} \bmod 3233 = 123$



RSA Algorithm

■ Python for RSA Generate Key Pair

```
from cryptography.hazmat.primitives.asymmetric import rsa
from cryptography.hazmat.primitives import serialization

# Generate private key
private_key = rsa.generate_private_key(public_exponent=65537, key_size=2048)

# Save private key
pem = private_key.private_bytes(
    encoding=serialization.Encoding.PEM,
    format=serialization.PrivateFormat.PKCS8,
    encryption_algorithm=serialization.NoEncryption()
)
with open("private_key.pem", "wb") as f:
    f.write(pem)
```



RSA Algorithm (Cont)

```
# Generate public key
public_key = private_key.public_key()

# Save public key
pem = public_key.public_bytes(
    encoding=serialization.Encoding.PEM,
    format=serialization.PublicFormat.SubjectPublicKeyInfo
)
with open("public_key.pem", "wb") as f:
    f.write(pem)
```



RSA Algorithm (Cont)

■ Python for RSA Encryption and Decryption

```
from cryptography.hazmat.primitives.asymmetric import padding
from cryptography.hazmat.primitives import hashes
message = b"Bank Wongyos"

# Encrypt
ciphertext = public_key.encrypt(
    message,
    padding.OAEP(mgf=padding.MGF1(algorithm=hashes.SHA256()), algorithm=hashes.SHA256(), label=None)
)

# Decrypt
plaintext = private_key.decrypt(
    ciphertext,
    padding.OAEP(mgf=padding.MGF1(algorithm=hashes.SHA256()), algorithm=hashes.SHA256(), label=None)
)

print("Decrypted:", plaintext.decode())
```



RSA Algorithm (Cont)

■ Python for RSA Cracking

```
from Crypto.Util.number import inverse, long_to_bytes

# Example weak RSA public key
n = 3233          # Public modulus (n = p * q)
e = 17           # Public exponent
cipher = 855      # Ciphertext to crack

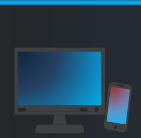
# Factor n manually (you know p=61, q=53 in this case)
p = 61
q = 53
phi = (p - 1) * (q - 1)
```



RSA Algorithm (Cont)

```
# Calculate private exponent
d = inverse(e, phi)

# Decrypt message:  $m = c^d \bmod n$ 
m = pow(cipher, d, n)
print("Decrypted message:", m)
print("As text:", long_to_bytes(m))
```



RSA Cracking Tools: CrypTool 2

 **CrypTool 2**
Cryptography for everybody

About CrypTool 2Documentation ▾DownloadsTeamVolunteerReport a Bug

CrypTool 2 Downloads

Here you can download the current versions of CrypTool 2.

There are two versions of CrypTool 2, the stable version and the nightly version. Both versions are available as an EXE installer and as a ZIP archive. The EXE installer supports the creation of a start menu entry, of a desktop link and of an Explorer file type. If you don't know which one to choose, you should prefer the stable version with EXE installer. No admin rights are needed for the installation. Each installation type (EXE and ZIP) has its own online update mechanism.

For execution, a 64-bit Windows and **Microsoft .NET Framework 4.7.2** or higher are needed.

Stable Version

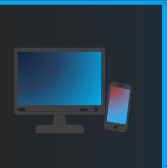
The 'stable version' is the CrypTool 2 **release** version. All components contained in this version have been extensively tested.

The current **release** version is **Setup CrypTool 2.1 (Stable Build 9778.2)** with the build date 01/31/2024 at 05:25:52 PM.

 **.exe Download**
Stable Version (170.55 MB)

 **.zip Download**
Stable Version (181.68 MB)

<https://www.cryptool.org/en/ct2/downloads/>



RSA Cracking Tools: FactorDB

[Search](#)[Sequences](#)[Report results](#)[Factor tables](#)[Status](#)[Downloads](#)[API](#)[Login](#)

Result:

status (?)	digits	number
FF	12 (show)	464654685152 _{<12>} = 2^5 · 7 · 281 · 7382033

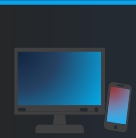
More information

ECM

factordb.com - 6 queries to generate this page (0.02 seconds) ([limits](#)) ([Privacy Policy / Imprint](#))

<https://factordb.com/>





RSA Cracking Tools: Alpertron

Integer factorization calculator

[Alpertron](#) > [Web applications](#) > Integer factorization calculator

Value

Actions:

Functions: Category:

Type one numerical expression or loop per line. Example: x=3;x=n(x);c<=100;x-1

Press the **Help** button to get help about this application. Press it again to return to the factorization. You can also watch [videos](#). Keyboard users can press CTRL+ENTER to start factorization. This is the WebAssembly version.

- $54165\ 489456\ 156156 = 2^2 \times 3^2 \times 13 \times 115\ 738225\ 333667$

Number of divisors: 36

<https://www.alpertron.com.ar/ECM.HTM>

Hashing





Overviews of Hashing

- **Hashing** or **message digest** is a cryptographic **hash function** containing a string of digits created by a **one-way hashing formula**
- **Hashing** is the process of converting input data of **any size** into a **fixed-size** string of characters, usually a sequence of numbers and letters
- The result is called a **hash value**, **hash code**, or **digest**

No key,
One-way methods



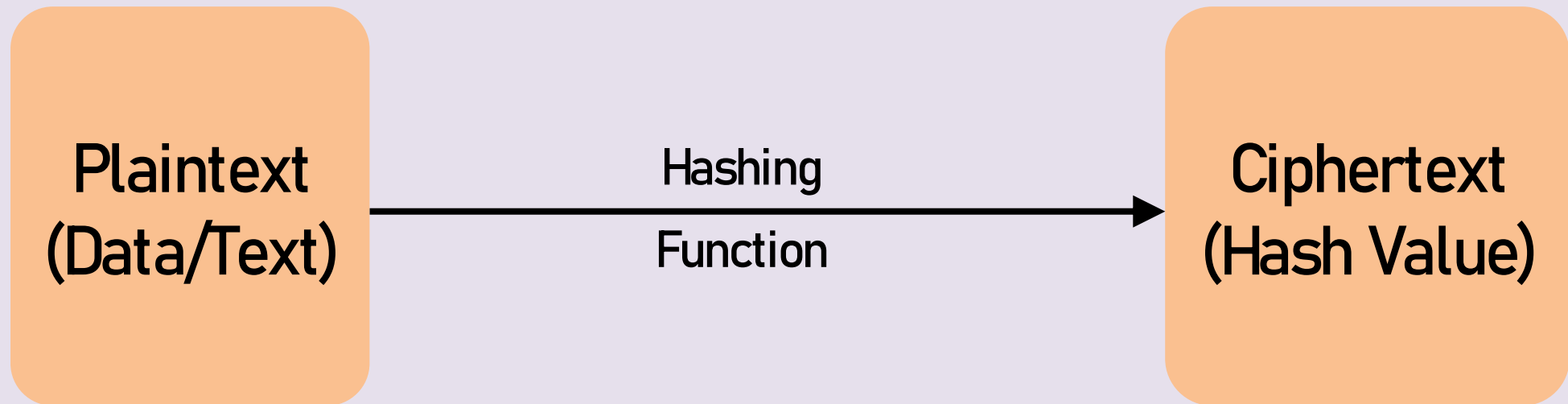
Key Concepts of Hashing

Term	Description
Hash Function	A mathematical algorithm that maps data to a fixed size
Deterministic	Same input always gives the same output
One-way	Practically impossible to reverse the process to get the original input
Collision-resistant	Hard to find two inputs that produce the same hash output



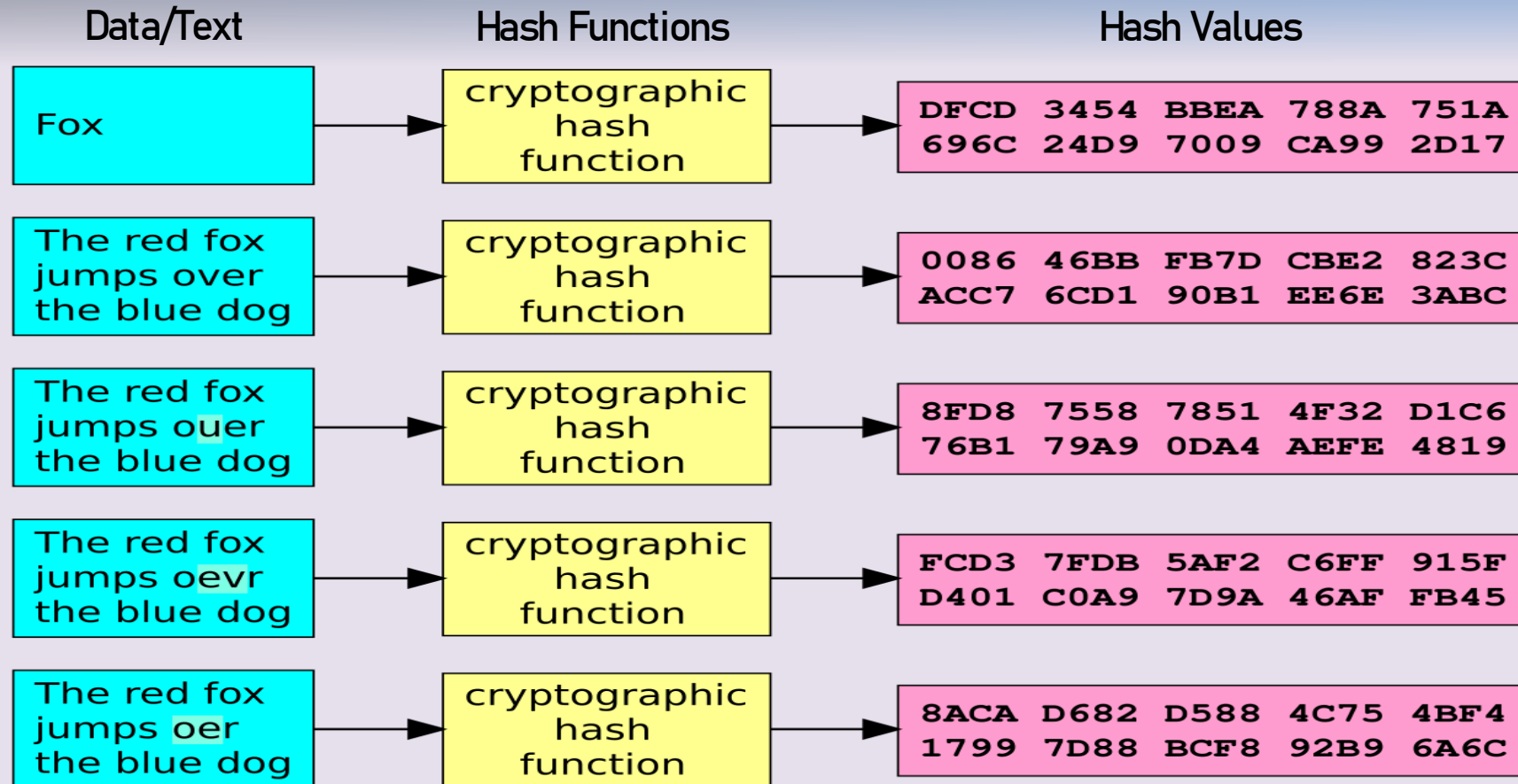


Process of Hashing





Process of Hashing (Cont)



https://en.wikipedia.org/wiki/Cryptographic_hash_function



Hashing Functions/Algorithms

Algorithm	Output Length	Description
MD5	128-bit	Fast but broken (not collision-resistant)
SHA-1	160-bit	More secure than MD5, but also broken
SHA-2	224/256/384/512-bit	Industry standard for many years (e.g., SHA-256)
SHA-3	224/256/384/512-bit	Next-gen cryptographic hash, designed by NIST
Bcrypt	184-bit	Slow by design to resist brute-force; includes salting and cost factor
BLAKE2	Variable	Faster than MD5/SHA but more secure
RIPEMD	128/160-bit	Alternative to MD5/SHA with similar use



MD5

- MD5 (Message Digest Algorithm 5) is a cryptographic hash function developed by Ronald Rivest in 1991
- It takes an input (message) and produces a 128-bit (16-byte) hash value, commonly represented as a 32-character hexadecimal string
- It is a deterministic function, meaning the same input will always produce the same hash
- Hash value can be cracked to:
 - Collision attacks (two different inputs producing the same hash)
 - Dictionary and rainbow table attacks





MD5 (Cont)

Recipe ^   

MD5 ^  

Input +   

Bank Wongyos|

RBC 12  1 **Tt** Raw Bytes

Output   

552bbf3eb86002082ac42098a3394680|



SHA-1

- **SHA-1 (Secure Hash Algorithm 1)** is a **cryptographic hash function** designed by the **NSA** and published by **NIST** in **1995**
- It produces a **160-bit** (20-byte) hash value, commonly expressed as a **40-character** hexadecimal string
- **SHA-1** was widely used for verifying data integrity, **digital signatures**, **certificates** (like SSL/TLS), and version control systems (e.g., Git)
- **SHA-1** is part of the original **SHA family** (SHA-0, SHA-1), later superseded by the more secure **SHA-2 family** (e.g., SHA-256) and **SHA-3**
- In 2017, the **SHAttered attack** demonstrated a **real-world collision**, making it unsuitable for secure applications



SHA-1 (Cont)

Recipe ^   

SHA1 ^  

Rounds
80

Input +    

Bank Wongyos  

ABC 12  1 **Tt** Raw Bytes  LF

Output     

e208a0c2a34453d4d406b60ae7207d62be07bba2



SHA-2

- **SHA-2** (Secure Hash Algorithm 2) is a family of **cryptographic hash functions** developed by the NSA and published by NIST in 2001 as an improvement over SHA-1
- **SHA-2** includes **six main hash functions**, based on digest lengths: SHA-224, SHA-256, SHA-384, SHA-512, SHA-512/224, and SHA-512/256
- The **most commonly used** version is **SHA-256**, which generates a **256-bit** (32-byte) hash, represented as a **64-character** hexadecimal string
- **Secure** and **recommended** for cryptographic use, **no practical collision**, preimage, or second preimage attacks currently known



SHA-2 (Cont)

Recipe ^   

SHA2 ^  

Size

512

Rounds

160

Input +    

Bank Wongyos  

ABC 12  1  Raw Bytes  LF

Output    

7bd3e1f4a33c8606193deafd4ce3c8eb1ca583906dc1870e
6151a0a3ad6d4c33282c45555d988e0b819237bfdb796714
df2ee60e84e5a8850032785d8861ec20



SHA-3

- **SHA-3** (Secure Hash Algorithm 3) is the **third** generation of the **SHA family**, officially released by **NIST** in **2015**
- **SHA-3** was developed **not as a replacement** for **SHA-2** (which remains secure), but as a cryptographically **distinct alternative**
- **SHA-3** supports **four fixed-length** output hash functions: SHA3-224, SHA3-256, SHA3-384, and SHA3-512
- It also introduces two **extendable-output functions** (XOFs): SHAKE128 and SHAKE256
- As of now, SHA-3 is **considered secure**, resistant to length-extension attacks, no successful practical collision, pre-image, or second pre-image attacks



SHA-3 (Cont)

Recipe ^

SHA3 ^

Size
512

Input +

Bank Wongyos

ABC 12 1 **Tt** Raw Bytes LF

Output

f1ec3c0e829ce550922dae2052b26cf4cf0bbc23d8fb1be1
3e9296e16cf84e7254c6f9043daf9f0f8ff749033ed855bc
401e8aae78c6dac88fa556b9863fd464



Bcrypt

- Bcrypt is a password hashing function designed for secure password storage
- It was introduced in 1999 by Niels Provos and David Mazières based on the Blowfish cipher
- Its design specifically addresses vulnerabilities found in standard hashing algorithms like MD5 and SHA by incorporating a work factor (cost) that makes it computationally expensive to brute-force
- Bcrypt is currently considered secure when configured with a sufficiently high cost factor (e.g., 12 or more) and resists brute-force and rainbow table attacks



Bcrypt (Cont)

Recipe ^

Bcrypt ^

Rounds
10

Input +

Bank Wongyos

ABC 12 1 Tt Raw Bytes LF

Output

\$2a\$10\$nKhtMq0vXXX.Pc0gcrTfo.WEpm447m8qEMig3GSAy
REJhJpC/a9Ai



BLAKE2

- **BLAKE2** is a **cryptographic hash function** designed as a **faster** and more **secure** alternative to **MD5**, **SHA-1**, and even **SHA-2**
- It was developed by **Jean-Philippe Aumasson** and others and is based on the **ChaCha** stream cipher's core
- It comes in **two** primary versions:
 - **BLAKE2b** – optimized for **64-bit** platforms (outputs up to **512 bits**)
 - **BLAKE2s** – optimized for **8-bit** to **32-bit** platforms (outputs up to **256 bits**)
- **BLAKE2** is not only fast, but also secure, and it has been **standardized** in **RFC 7693**



BLAKE2 (Cont)

Recipe ^

BLAKE2b ^

Size

512

Output Encoding

Hex

Key

UTF8 ▾

Input +

Bank Wongyos

ABC 12 1 Tt Raw Bytes LF

Output

7e6c6bb20509eaf61aa4ac9bb575694c4d89e589cc8ab0e5
7e1a730be05bdf2ffad54fae06c9d959490d7c03b1af7fea
e8a1963b5cbef724e918ae727d70945b



RIPEMD

- RIPEMD is a family of **cryptographic hash functions** developed in **1996** by **Hans Dobbertin**, **Antoon Bosselaers**, and **Bart Preneel** at the Katholieke Universiteit Leuven in Belgium
- There are multiple variants of RIPEMD: RIPEMD-128, **RIPEMD-160** (**most commonly used**), RIPEMD-256, and RIPEMD-320
- **RIPEMD-160** is considered **secure** and has not been practically broken but **RIPEMD-128** has similar vulnerabilities to MD4 and MD5, and is **not recommended** for secure applications
- RIPEMD still used in certain **blockchain** and cryptographic applications (e.g., **Bitcoin addresses** use SHA-256 followed by RIPEMD-160)



RIPEMD (Cont)

Recipe ^   

RIPEMD ^  

Size

320

Input +    

Bank Wongyos  

ABC 12  1 **Tt** Raw Bytes  LF

Output    

fe40c396b5c76e08e71e7b1013b8b0bf7d413b264164734a
42c9f55b9dae1798ae597387e03b747d



Hash Cracking

- **Hash cracking** is the process of **finding the original input** of a hashed value by attempting to **reverse** it — often to test or break password security
- Although hash functions are designed to be **one-way**, attackers can still try to **"crack"** them using **various techniques**
- Hash cracking should only be done in:
 - **Penetration testing** with explicit permission
 - Security **research**
 - **Educational** and **learning** environments
- **Unauthorized use** is **illegal** and **unethical**



Hash Cracking (Cont)

- Hash Cracking **Methods**

Method	Description
Brute Force	Try all possible combinations until a match is found
Dictionary Attack	Use a list of common passwords or words as input
Rainbow Table Attack	Use precomputed hash values for faster lookups
Hybrid Attack	Combine dictionary and brute-force (e.g., add numbers or symbols to dictionary words)
Rule-based Attack	Apply transformations to dictionary words (e.g., leetspeak, capitalizations)



Hash Cracking (Cont)

Method	Description
Mask Attack	Use patterns like Passw??d123 to reduce search space
Lookup in Online Databases	Search for the hash in online databases like CrackStation or Hashes.com



Hash Cracking (Cont)

■ Popular Tools for Hash Cracking

Tool	Description
Hashcat	World's fastest and most advanced hash cracker; supports GPU acceleration
John the Ripper	Powerful and customizable password cracker. Can crack hashes from many formats
Hydra	Often used for password attacks over network protocols, but can do hash cracking with external scripts
RainbowCrack	Tool for generating and using rainbow tables



Hash Cracking (Cont)

Tool	Description
CrackStation	Online hash cracker with large precomputed dictionary
Hashes.com	A hash lookup service allows to input hash value and search for its corresponding plaintext
Cain & Abel	Legacy GUI-based hash cracker for Windows (deprecated, but still referenced)



Hash Cracking (Cont)

■ Hashcat Examples

```
hashcat -m [hash-mode] -a [attack-mode] -o [output-file] [hash-file]  
[wordlist]
```

#Crack MD5 with Dictionary Attack

```
hashcat -m 0 -a 0 hashes.txt rockyou.txt
```

#Crack SHA-1 Hash

```
hashcat -m 100 -a 0 hashes.txt wordlist.txt
```

#Mask Attack (Brute Force)

```
hashcat -m 0 -a 3 hash.txt ?a?a?a?a?a?
```

#Crack bcrypt

```
hashcat -m 3200 -a 0 bcrypt_hashes.txt wordlist.txt
```



Hash Cracking (Cont)

■ John the Ripper Examples

```
john [options] [hashfile]
```

Create a file **hashes.txt**
with **one hash per line**

```
#Run John with Default Wordlist
```

```
john hashes.txt
```

```
#Use a Custom Wordlist
```

```
john --wordlist=/usr/share/wordlists/rockyou.txt hashes.txt
```

```
#Show Cracked Passwords
```

```
john --show hashes.txt
```

```
#Check Hash Type
```

```
john --list=formats
```

```
john --format=raw-md5 hashes.txt
```



Hash Cracking (Cont)

- CrackStation Examples: <https://crackstation.net/>

The screenshot shows the CrackStation website. At the top, there's a navigation bar with the 'CrackStation' logo on the left and 'Defuse.ca' with a Twitter icon on the right. Below the navigation bar, there's a dropdown menu with 'CrackStation', 'Password Hashing Security', and 'Defuse Security'. The main heading is 'Free Password Hash Cracker'. Below this, it says 'Enter up to 20 non-salted hashes, one per line:' followed by a large text input area. To the right of the input area is a reCAPTCHA widget with the text 'I'm not a robot' and a 'Crack Hashes' button. Below the input area, there's a list of supported hash types: 'Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-half, sha1, sha224, sha256, sha384, sha512, ripeMD160, whirlpool, MySQL 4.1+ (sha1 sha1_bin)), QubesV3.1BackupDefaults'. At the bottom, there's a link to 'Download CrackStation's Wordlist'.



Hash Cracking (Cont)

- Hashes.com Examples: <https://hashes.com/en/decrypt/hash>

The screenshot shows the Hashes.com website interface. At the top is a dark blue navigation bar with the site logo and various links. The main content area has a dark blue header with the text 'Decrypt MD5, SHA1, MySQL, NTLM, SHA256, MD5 Email, SHA256 Email, SHA512 hashes'. Below this is a grey box with instructions: 'Enter your hashes here and we will attempt to decrypt them for free online.' and 'Hashes (max. 25 separated by newline, format 'hash[:salt]') (QEscrow)'. A large text input field with the placeholder 'Write here...' is provided. At the bottom of the form, there are two checkboxes: 'Show plains and salts in hex format' and 'Show algorithm of founds'. To the right of these checkboxes is a captcha image showing the text 'pTsf9' and a 'Captcha Check' button.

Hashes.com

Home ? FAQ Deposit to Escrow Purchase Credits API Tools Decrypt Hashes Escrow ?Support English Register Login

Decrypt MD5, SHA1, MySQL, NTLM, SHA256, MD5 Email, SHA256 Email, SHA512 hashes

Enter your hashes here and we will attempt to decrypt them for free online.

Hashes (max. 25 separated by newline, format 'hash[:salt]') (QEscrow)

Write here...

☐ Show plains and salts in hex format ☐ Show algorithm of founds



Q & A

Asst.Prof.Pol.Lt.Col.Dr. Wongyos Keardsri (Bank)

Lecturer (Level 3) | Faculty of Forensic Science | Royal Police Cadet Academy | Thailand

Email: wongyos@gmail.com | Line ID: wongyos | Phone/WhatsApp: +6689-599-3490

Facebook: facebook.com/wongyos



End